

Отчёт по лабораторной работе №5

Компьютерный практикум по статистическому анализу данных

Построение графиков

Выполнил: Исаев Булат Абубакарович,
НПИбд-01-22, 1132227131

Содержание

1	Цель работы	1
2	Выполнение лабораторной работы.....	2
2.1	Основные пакеты для работы с графиками в Julia.....	2
2.2	Опции при построении графика	3
2.3	Точечный график.....	8
2.4	Аппроксимация данных	10
2.5	Две оси ординат.....	12
2.6	Полярные координаты	12
2.7	Параметрический график	13
2.8	График поверхности	15
2.9	Линии уровня	19
2.10	Векторные поля.....	22
2.11	Анимация	23
2.12	Гипоциклоида.....	25
2.13	Errorbars	30
2.14	Использование пакета Distributions	35
2.15	Подграфики	39
2.16	Самостоятельное выполнение	44
3	Вывод.....	56
4	Список литературы. Библиография	56

1 Цель работы

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

2 Выполнение лабораторной работы

2.1 Основные пакеты для работы с графиками в Julia

Julia поддерживает несколько пакетов для работы с графиками. Использование того или иного пакета зависит от целей, преследуемых пользователем при построении. Стандартным для Julia является пакет Plots.jl.

Рассмотрим построение графика функции $f(x) = (3x^2 + 6x - 9)e^{-0,3x}$ разными способами (рис. 1 - (рис. 2):

```
[2]: using Plots

# задание функции:
f(x) = (3x.^2 + 6x .- 9).*exp.(-0.3x)
# генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1
# (шаг задан через указание длины массива):
x = collect(range(-5,10,length=151))
# генерирование массива значений y:
y = f(x)
# указывается, что для построения графика используется gr():
gr()

# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,
title="A simple curve",
xlabel="Variable x",
ylabel="Variable y",
color="blue")
```

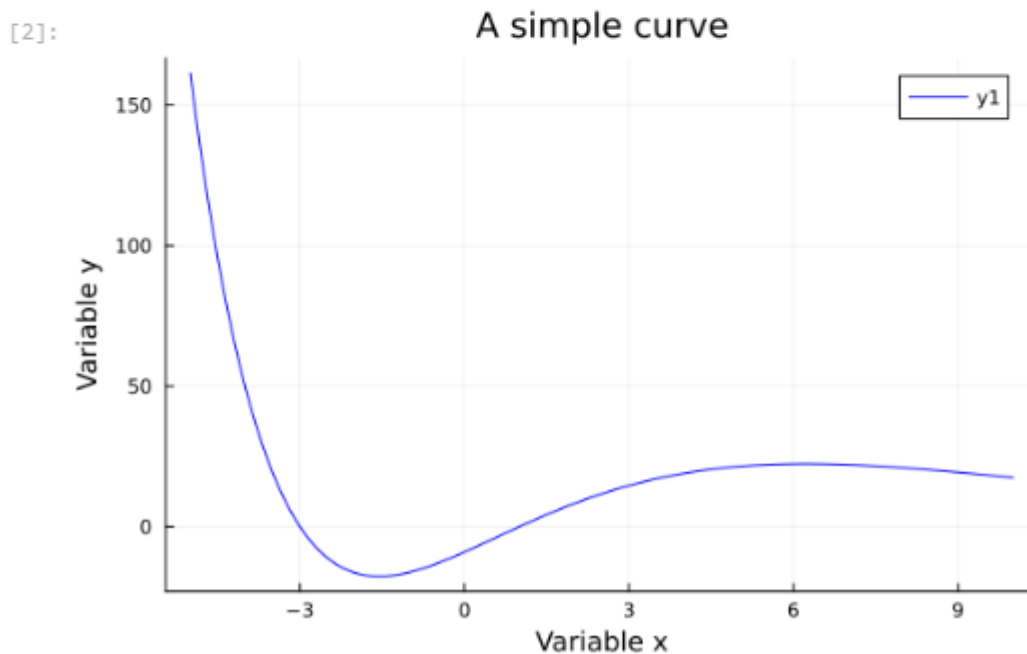


Рис. 1: График функции, построенный при помощи `gr()`

```
[11]: using Plots

# Используем GR (работает без Python)
gr()

# Определяем функцию и данные
f(x) = (3x.^2 + 6x - 9).*exp.(-0.3x)
x = range(-5, 10, length=151)
y = f(x)

# Строим график
plot(x, y,
      title="Simple curve",
      xlabel="Variable x",
      ylabel="Variable y",
      label="f(x) = (3x² + 6x - 9)e^(-0.3x)",
      color="blue",
      lw=2,
      grid=true
    )
```

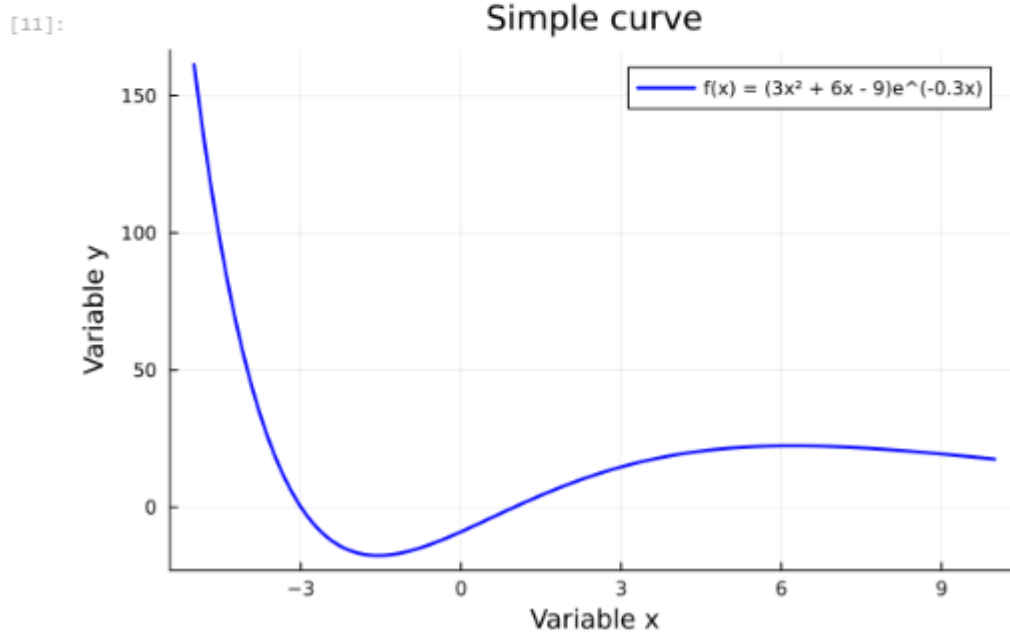


Рис. 2: График функции, построенный при помощи pyplot()

2.2 Опции при построении графика

На примере графика функции $\sin(x)$ и графика разложения этой функции в ряд Тейлора рассмотрим дополнительные возможности пакетов для работы с графикой (рис. 3 - рис. 5):

```
[16]: using Plots
      gr() # Используем GR вместо PyPlot

      # задание функции sin(x):
      sin_theor(x) = sin(x)

      # построение графика функции sin(x):
      plot(sin_theor,
           title="График функции sin(x)",
           xlabel="Ось x - аргумент",
           ylabel="Ось y - значение функции",
           label="sin(x)",
           color="blue",
           linewidth=2,
           grid=true)
```

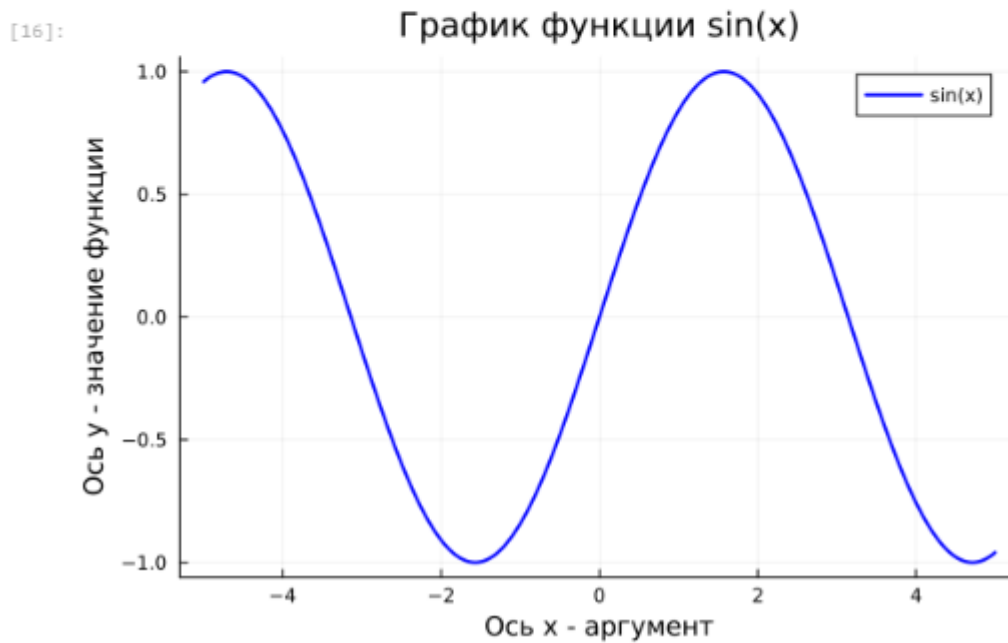


Рис. 3: График функции $\sin(x)$

```
[18]: using Plots
      gr()

      # задание функции sin(x):
      sin_theor(x) = sin(x)

      # задание функции разложения исходной функции в ряд Тейлора:
      sin_taylor(x) = [(-1)^i*x^(2*i+1)/factorial(2*i+1) for i in 0:4] |> sum

      # построение графика функции sin_taylor(x):
      plot(sin_taylor,
           title="Разложение sin(x) в ряд Тейлора",
           xlabel="x",
           ylabel="sin_taylor(x)",
           label="Ряд Тейлора (5 членов)",
           color="red",
           linewidth=2,
           grid=true)
```

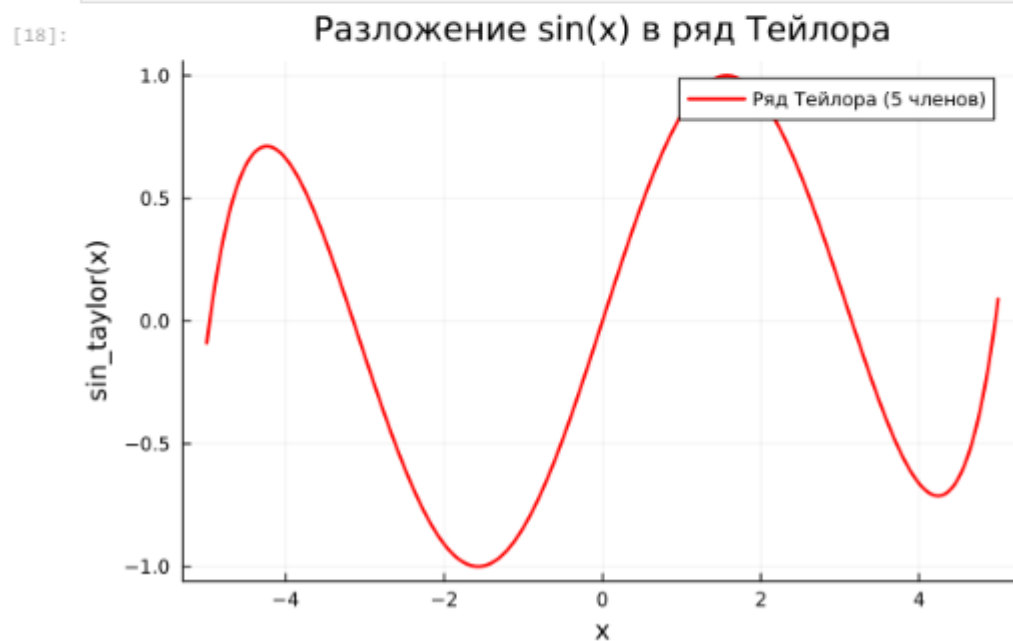


Рис. 4: График функции разложения исходной функции в ряд Тейлора

```
[22]: # построение графика функции sin_taylor(x):
plot(sin_theor,
     label="sin(x)",
     color="blue",
     grid=true)

# построение графика функции sin_taylor(x):
plot!(sin_taylor,
     label="Ряд Тейлора (5 членов)",
     color="red",
     grid=true)

title!("Сравнение двух функций")
xlabel!("Ось x (аргумент)")
ylabel!("Ось y (значение функции)")
```

[22]:

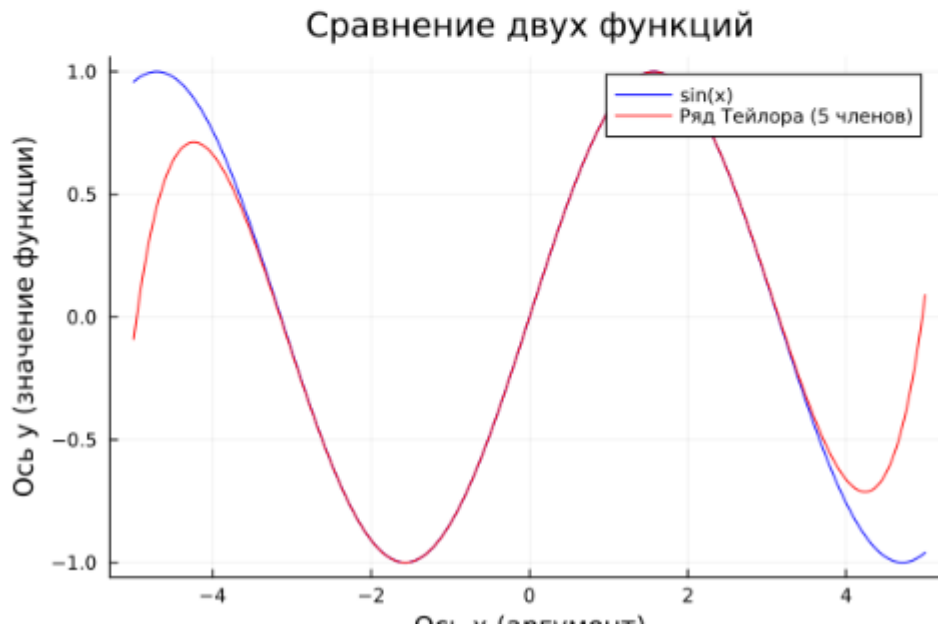


Рис. 5: Графики исходной функции и её разложения в ряд Тейлора

Затем добавим различные опции для отображения на графике (рис. 6):

```
[19]: plot(
    # функция  $\sin(x)$ :
    sin_taylor,
    # подпись в легенде, цвет и тип линии:
    label = "sin(x), разложение в ряд Тейлора",
    line=(blue, 0.3, 6, :solid),
    # размер графика:
    size=(800, 500),
    # параметры отображения значений по осям
    xticks = (-5:0.5:5),
    yticks = (-1:0.1:1),
    xtickfont = font(12, "Times New Roman"),
    ytickfont = font(12, "Times New Roman"),

    # подписи по осям:
    ylabel = "y",
    xlabel = "x",
    # название графика:
    title = "Разложение в ряд Тейлора",
    # поворот значений, заданный по оси x:
    xrotation = rad2deg(pi/4),
    # заливка области графика цветом:
    fillrange = 0,
    fillalpha = 0.5,
    fillcolor = :lightgoldenrod,
    # задание цвета фона:
    background_color = :ivory)

    plot!()
    # функция sin_theor:
    sin_theor,
    # подпись в легенде, цвет и тип линии:
    label = "sin(x), теоретическое значение",
    line=(black, 1.0, 2, :dash))
```



Рис. 6: Вид графиков после добавления опций при их построении

2.3 Точечный график

Как и при построении обычного графика для точечного графика необходимо задать массив значений x , посчитать или задать значения y , задать опции построения графика (рис. 7):

```
[25]: # параметры распределения точек на плоскости:
```

```
x = range(1,10,length=10)
```

```
y = rand(10)
```

```
# параметры построения графика:
```

```
plot(x, y,  
      seriestype = :scatter,  
      title = "Точечный график",  
      xlabel="Ось X ('x')",  
      ylabel="Ось Y (случайные значения)",  
      label="Точки данных",  
      color="blue",  
      markersize=10,  
      grid=true)
```

```
[25]:
```

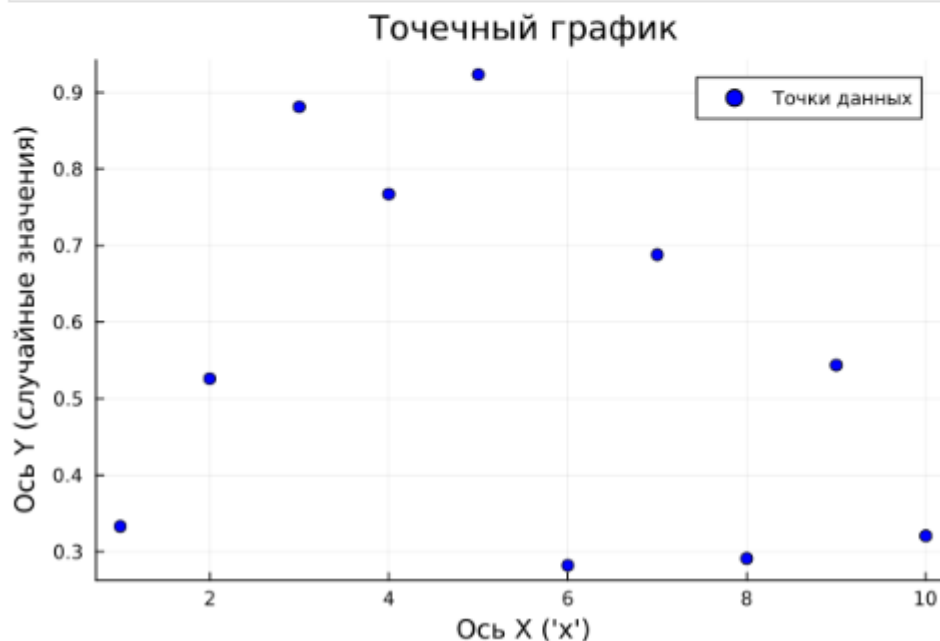


Рис. 7: График десяти случайных значений на плоскости (простой точечный график)

Для точечного графика можно задать различные опции, например размер маркера, его тип, цвет и и т.п. (рис. 8):


```
[30]: n = 50  
x = rand(n)  
y = rand(n)  
ms = rand(50) * 30  
scatter(x, y, markersize=ms)
```

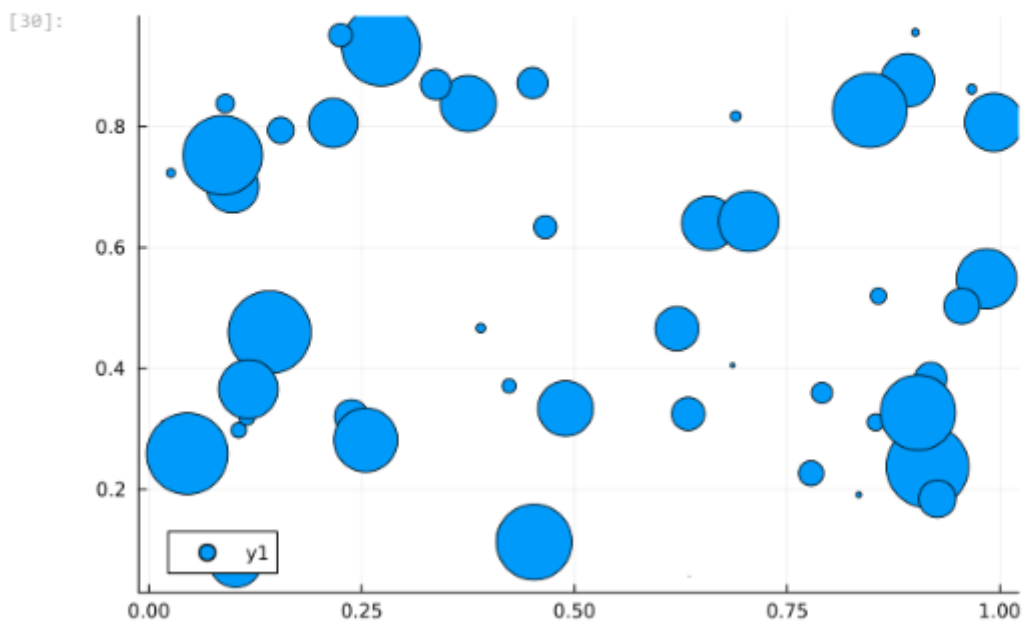


Рис. 8: График пятидесяти случайных значений на плоскости с различными опциями отображения (точечный график с кодированием значения размером точки)

Также можно строить и 3-мерные точечные графики (рис. 9):

```
[31]: n = 50  
x = rand(n)  
y = rand(n)  
z = rand(n)  
ms = rand(50) * 30  
scatter(x, y, z, markersize=ms)
```

[31]:

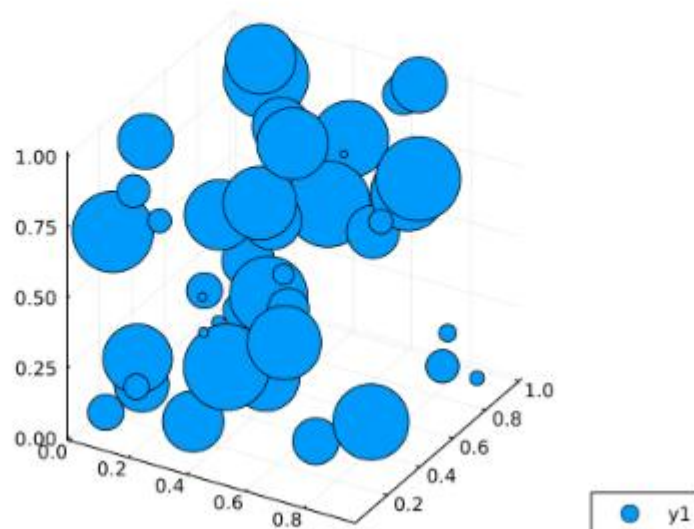


Рис. 9: График пятидесяти случайных значений в пространстве с различными опциями отображения (3-мерный точечный график с кодированием значения размером точки)

2.4 Аппроксимация данных

Аппроксимация — научный метод, состоящий в замене объектов их более простыми аналогами, сходными по своим свойствам.

Для демонстрации зададим искусственно некоторую функцию, в данном случае похожую по поведению на экспоненту (рис. 10):

```
[32]: x = collect(0:0.01:9.99)
      y = exp.(ones(1000)+x) + 4000*randn(1000)
      scatter(x,y,markersize=3,alpha=.8,legend=false)
```

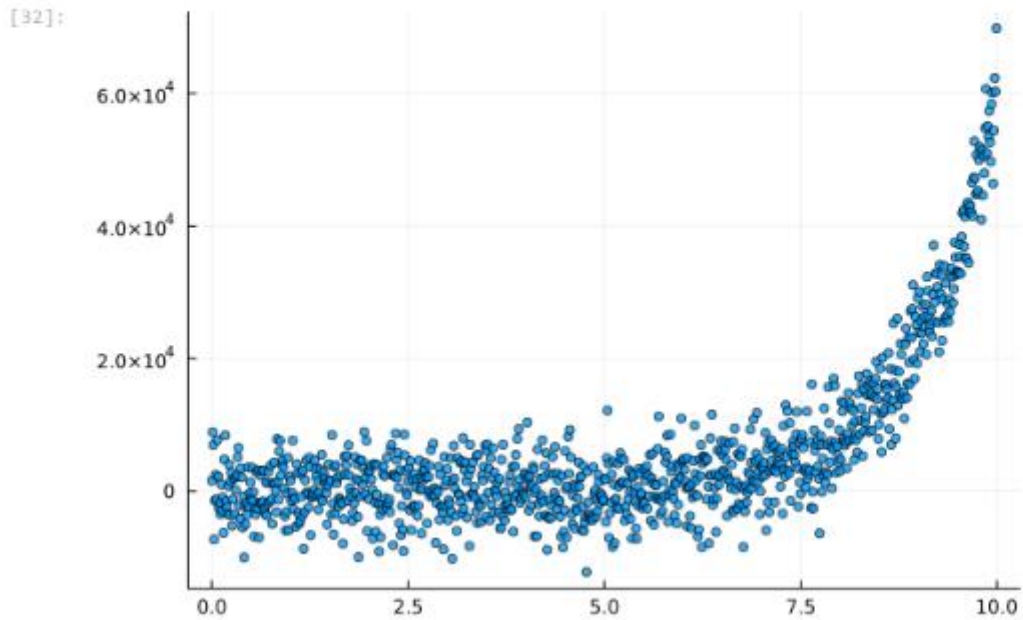


Рис. 10: Пример функции

Аппроксимируем полученную функцию полиномом 5-й степени (рис. 11):

```
[36]: A = [ones(1000) x x.^2 x.^3 x.^4 x.^5]
      c = A \ y
      f_poly = c[1]*ones(1000) + c[2]*x + c[3]*x.^2 + c[4]*x.^3 + c[5]*x.^4 + c[6]*x.^5
      plot!(x, f_poly, linewidth=3, color=:red)
```

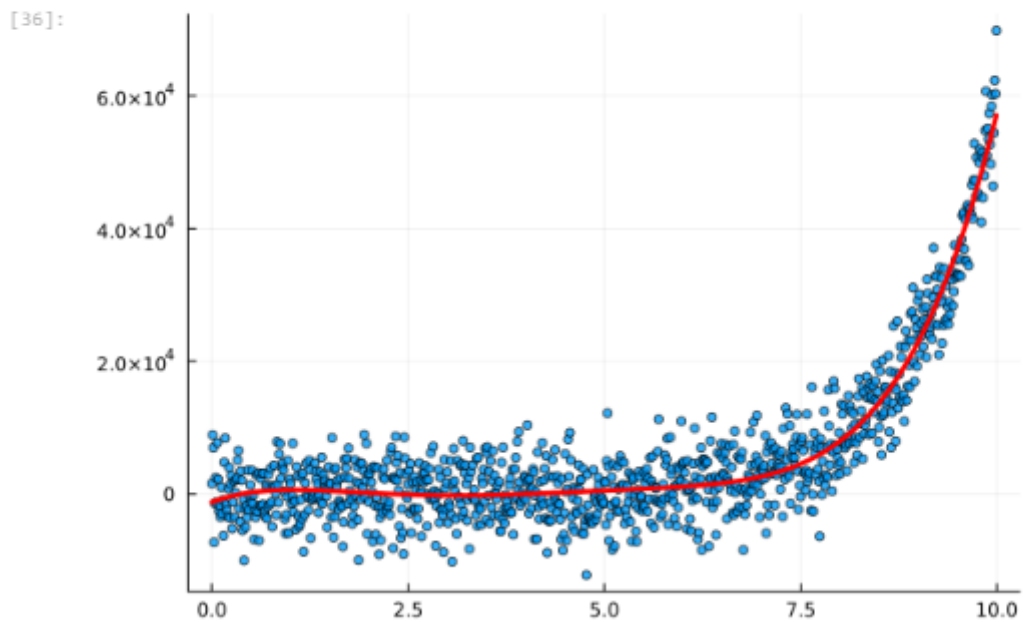


Рис. 11: Пример аппроксимации исходной функции полиномом 5-й степени

2.5 Две оси ординат

Иногда требуется на один график вывести несколько траекторий с существенными отличиями в значениях по оси ординат.

Пример первой траектории (рис. 12):

```
[37]: plot(randn(100), ylabel="y1", leg=:topright, grid=:off)
```

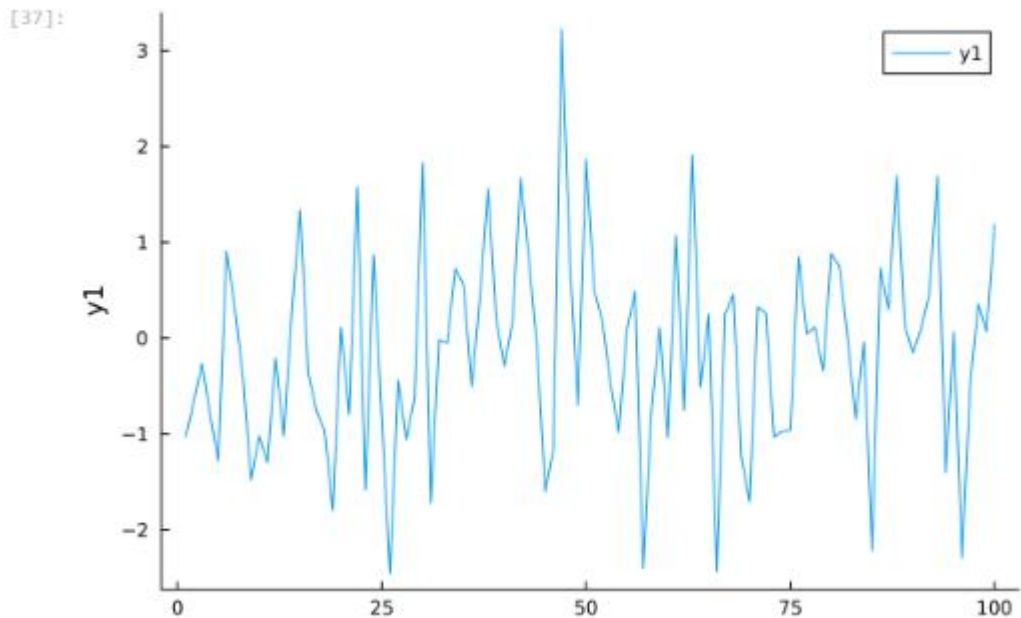


Рис. 12: Пример отдельно построенной траектории

2.6 Полярные координаты

Приведём пример построения графика функции в полярных координатах (рис. 13):

```
[40]: # функция в полярных координатах:
r(θ) = 1 + cos(θ) * sin(θ)^2
# полярная система координат:
θ = range(0, stop=2π, length=50)
# график функции, заданной в полярных координатах:
plot(θ, r.(θ),
proj=:polar,
lims=(0,1.5))
```

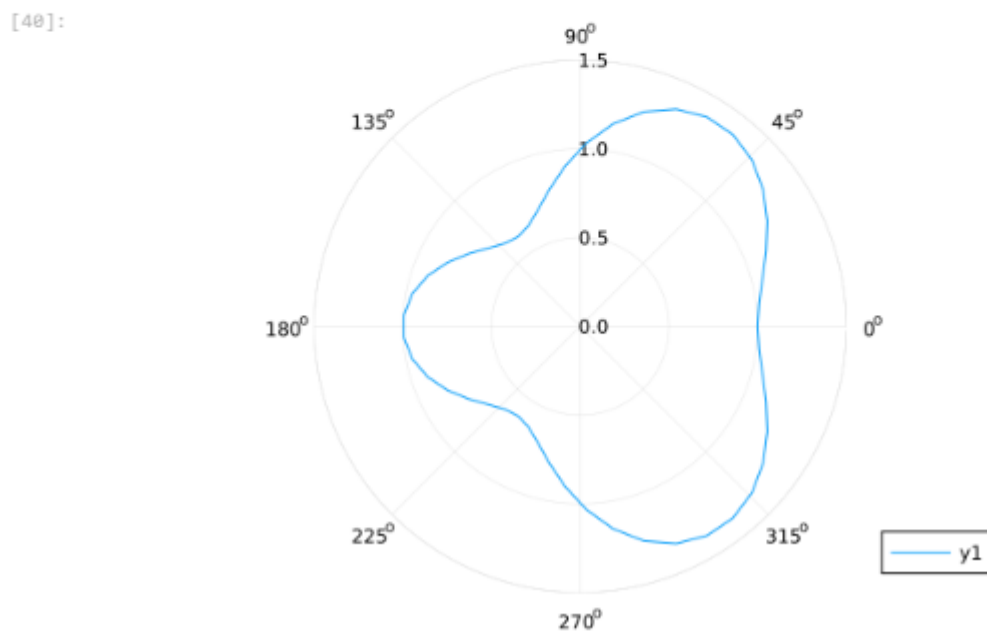


Рис. 13: График функции, заданной в полярных координатах

2.7 Параметрический график

Приведём пример построения графика параметрически заданной кривой на плоскости (рис. 14):

```
[46]: xfun(t) = sin(t)
      yfun(t) = sin(2t)

      plot(xfun, yfun, 0, 2π, leg=false, fill=(0, :orange))
```

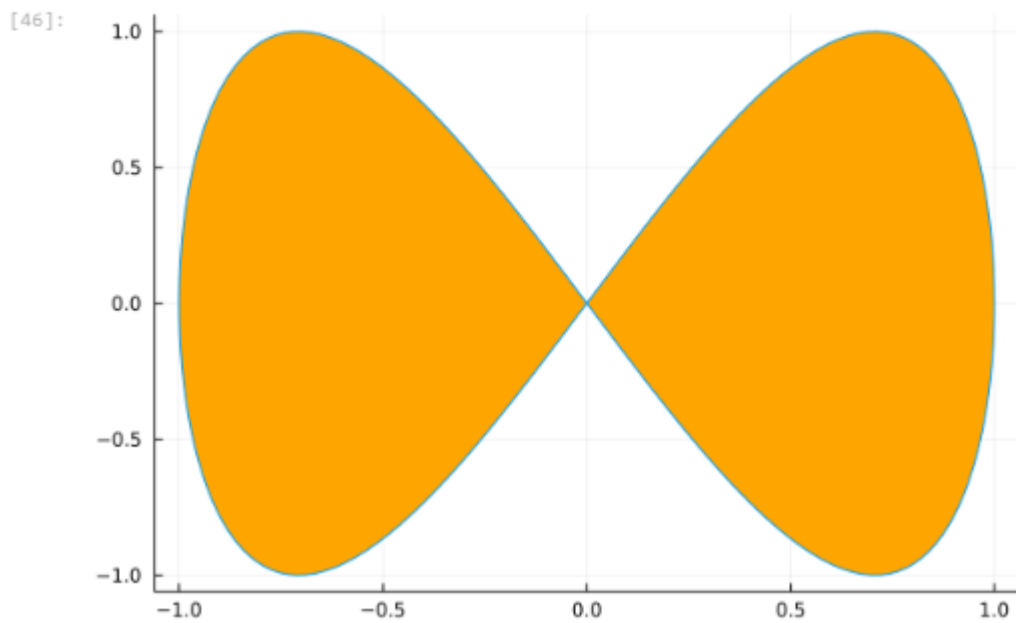


Рис. 14: Параметрический график кривой на плоскости

Далее приведём пример построения графика параметрически заданной кривой в пространстве (рис. 15):

```
[48]: t = range(0, stop=10, length=1000)
      x = cos.(t)
      y = sin.(t)
      z = sin.(5t)
      plot(x, y, z)
```

[48]:

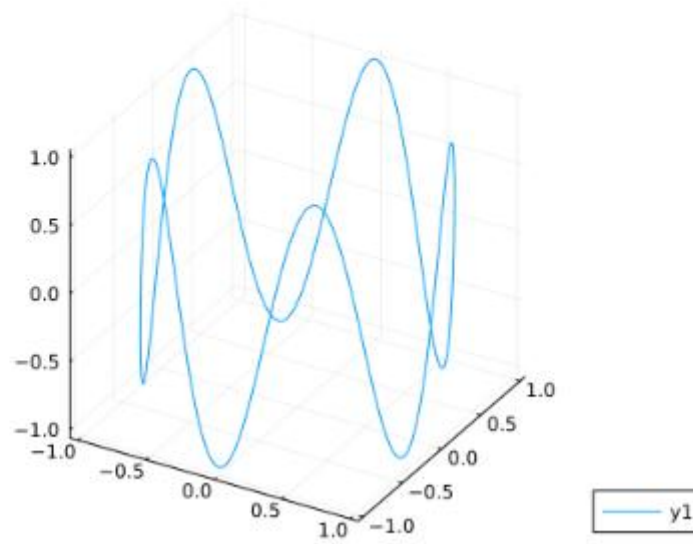


Рис. 15: Параметрический график кривой в пространстве

2.8 График поверхности

Для построения поверхности, заданной уравнением $f(x, y) = x^2 + y^2$, можно воспользоваться функцией `surface()` (рис. 16):

```
[49]: f(x,y) = x^2 + y^2  
      x = -10:0.1:10  
      y = x  
      plot(x, y, f,  
           linestyle = :surface)
```

[49]:

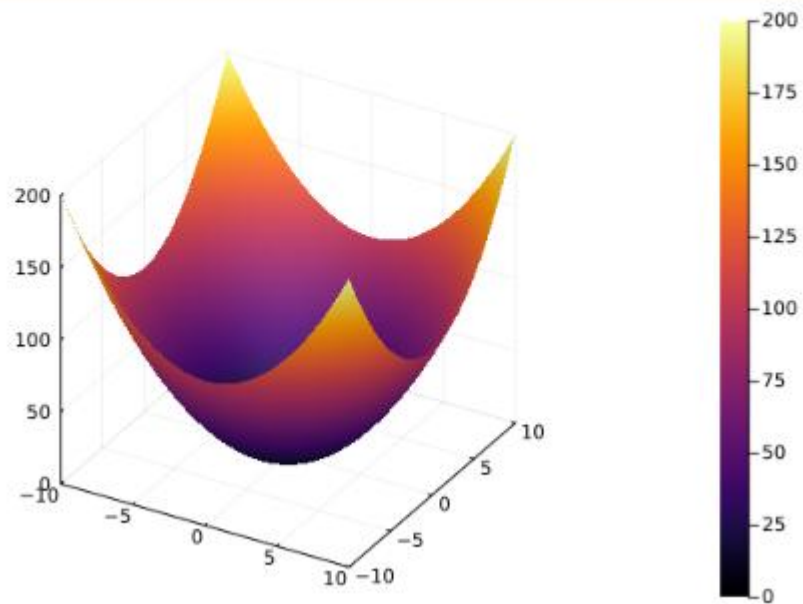


Рис. 16: График поверхности (использована функция `surface()`)

Также можно воспользоваться функцией `plot()` с заданными параметрами (рис. 17):


```
[53]: f(x,y) = x^2 + y^2  
x = -10:10  
y = x  
plot(x, y, f,  
      linetype = :wireframe)
```

[53]:

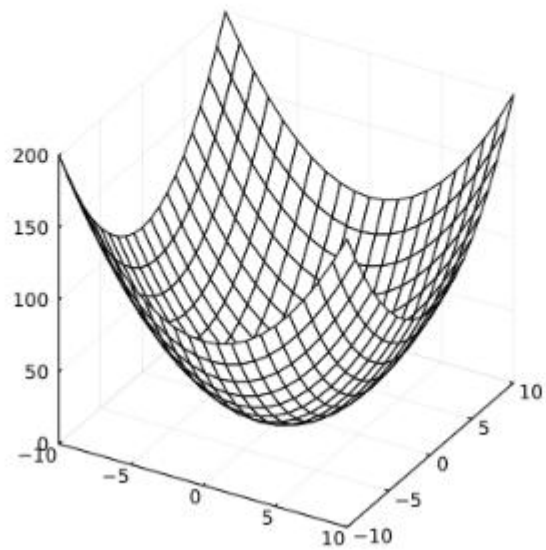


Рис. 17: График поверхности (использована функция `plot()`)

Можно задать параметры сглаживания (рис. 18):

```
[62]: f(x,y) = x^2 + y^2  
x = -10:0.1:10  
y = x  
plot(x, y, f,  
      linestyle = :surface,  
      color = :viridis,  
      legend = false,  
      colorbar = true)
```

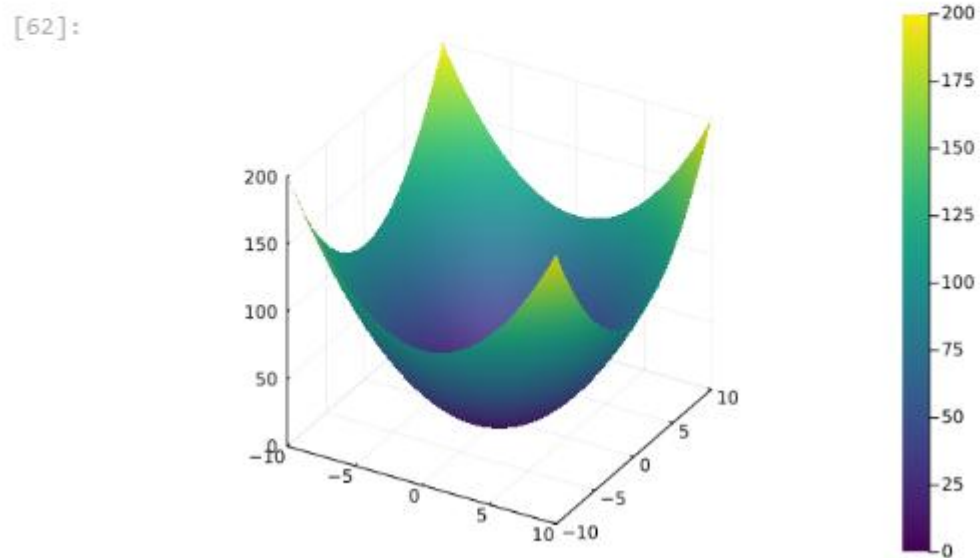


Рис. 18: Сглаженный график поверхности

Можно задать определённый угол зрения (рис. 19):

```
[68]: x=range(-2,stop=2,length=100)
      y=range(sqrt(2),stop=2,length=100)
      f(x,y) = x*y-x-y+1
      plot(x,y,f,
           linestyle = :surface,
           c=cgrad([:red,:blue]),
           camera=(-30,30),
           )
```

[68]:

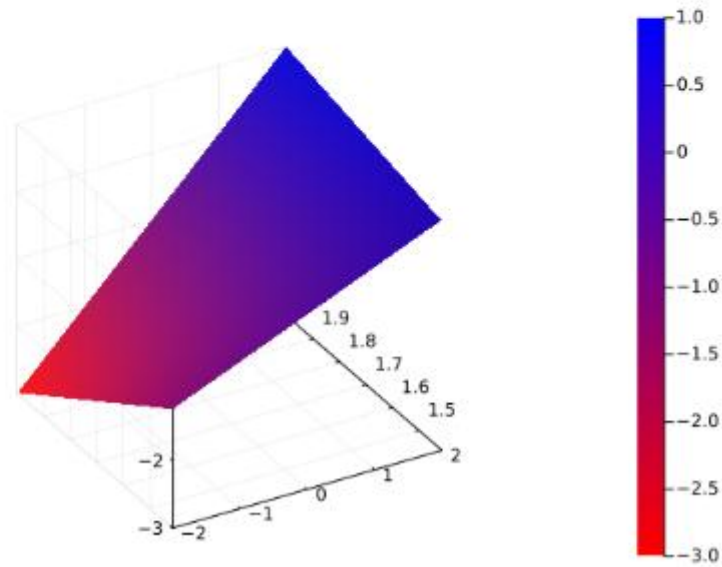


Рис. 19: График поверхности с изменённым углом зрения

2.9 Линии уровня

Линией уровня некоторой функции от двух переменных называется множество точек на координатной плоскости, в которых функция принимает одинаковые значения. Линий уровня бесконечно много, и через каждую точку области определения можно провести линию уровня.

С помощью линий уровня можно определить наибольшее и наименьшее значение исходной функции от двух переменных. Каждая из этих линий соответствует определённому значению высоты.

Поверхности уровня представляют собой непересекающиеся пространственные поверхности.

Рассмотрим поверхность, заданную функцией $g(x, y) = (3x + y^2) | \sin(x) + \cos(y) |$ (рис. 20):

```

•[70]: x = 1:0.5:20
      y = 1:0.5:10
      g(x, y) = (3x + y ^ 2) * abs(sin(x) + cos(y))
      plot(x,y,g,|
          linestyle = :surface,)

```

[70]:

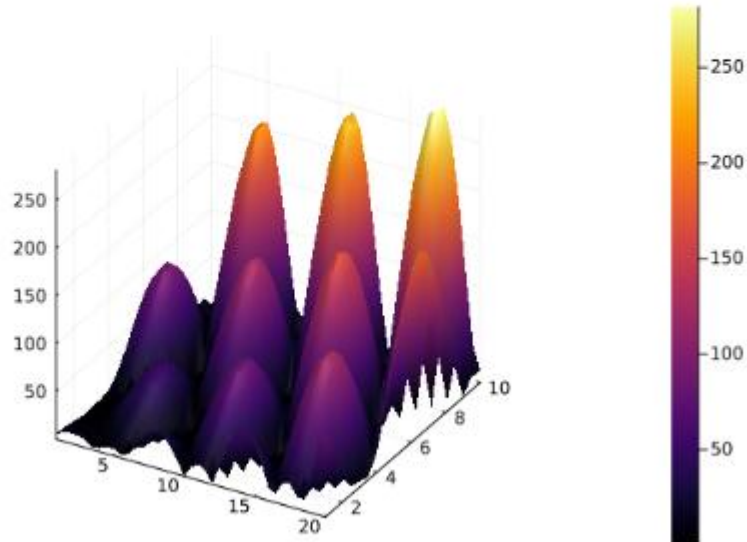


Рис. 20: График поверхности, заданной функцией $g(x, y) = (3x + y^2) \cdot |\sin(x) + \cos(y)|$

Линии уровня можно построить, используя проекцию значений исходной функции на плоскость (рис. 21):

```
[73]: p = contour(x, y, g)
      plot(p)
```

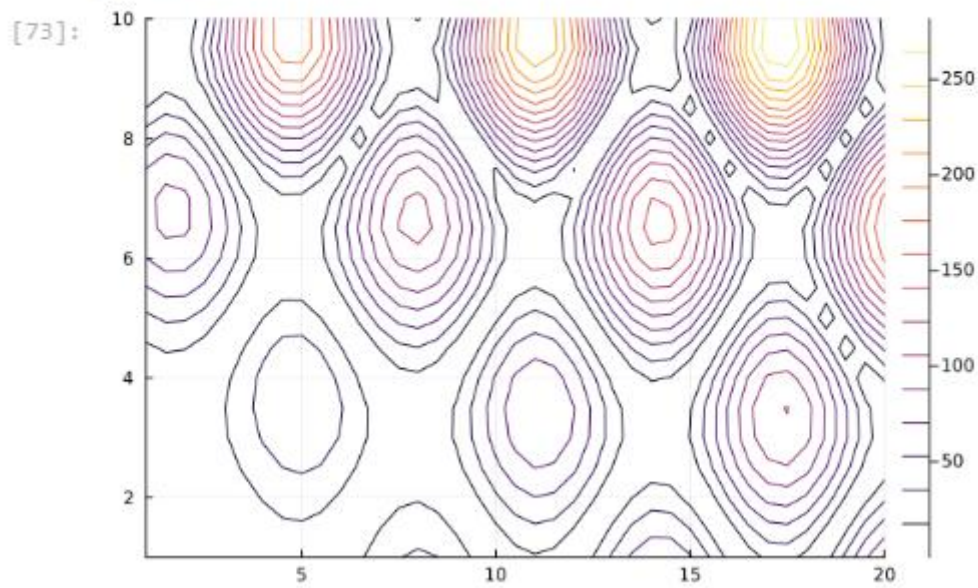


Рис. 21: Линии уровня

Можно дополнительно добавить заливку цветом (рис. 22):

```
[74]: p = contour(x, y, g,
      fill=true)
      plot(p)
```

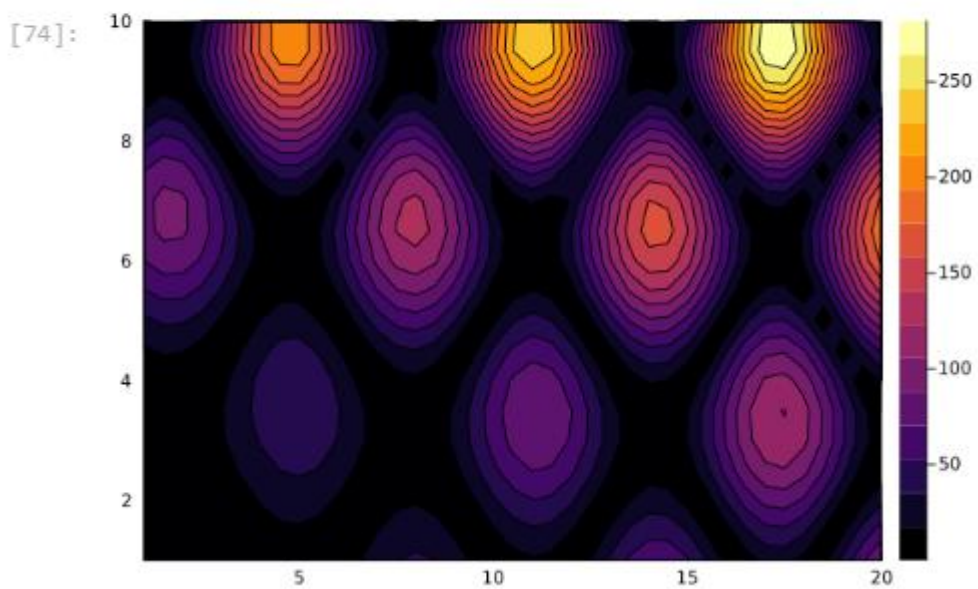


Рис. 22: Линии уровня с заполнением

2.10 Векторные поля

Если каждой точке некоторой области пространства поставлен в соответствие вектор с началом в данной точке, то говорят, что в этой области задано векторное поле.

Векторные поля задают векторными функциями.

Для функции $h(x, y) = x^3 - 3x + y^2$ сначала построим её график (рис. 23) и линии уровня (рис. 24):

```
[89]: x = range(-2, stop=2, length=100)
      y = range(-2, stop=2, length=100)

      h(x, y) = x^3 - 3x + y^2

      plot(X, Y, h,
           linestyle = :surface)
```

[89]:

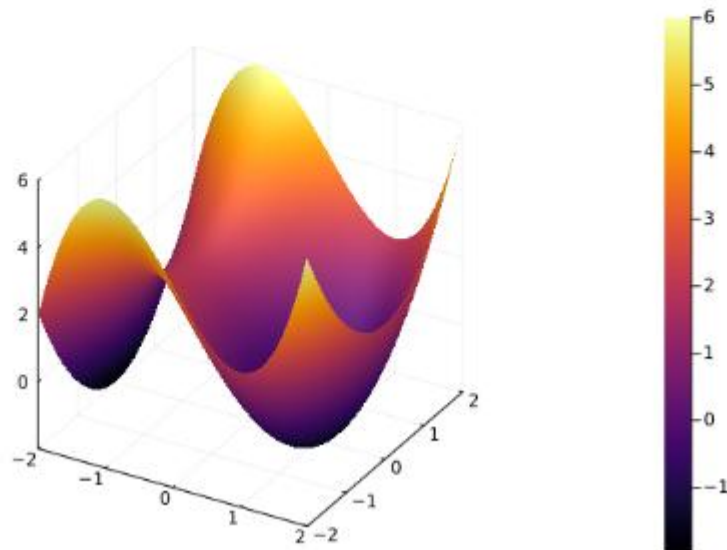


Рис. 23: График функции $h(x, y) = x^3 - 3x + y^2$

```
[87]: # определение переменных:
X = range(-2, stop=2, length=100)
Y = range(-2, stop=2, length=100)
# определение функции:
h(x, y) = x^3 - 3x + y^2
# построение поверхности:
plot(X,Y,h,
linetype = :surface)
# построение линий уровня:
contour(X, Y, h)
```

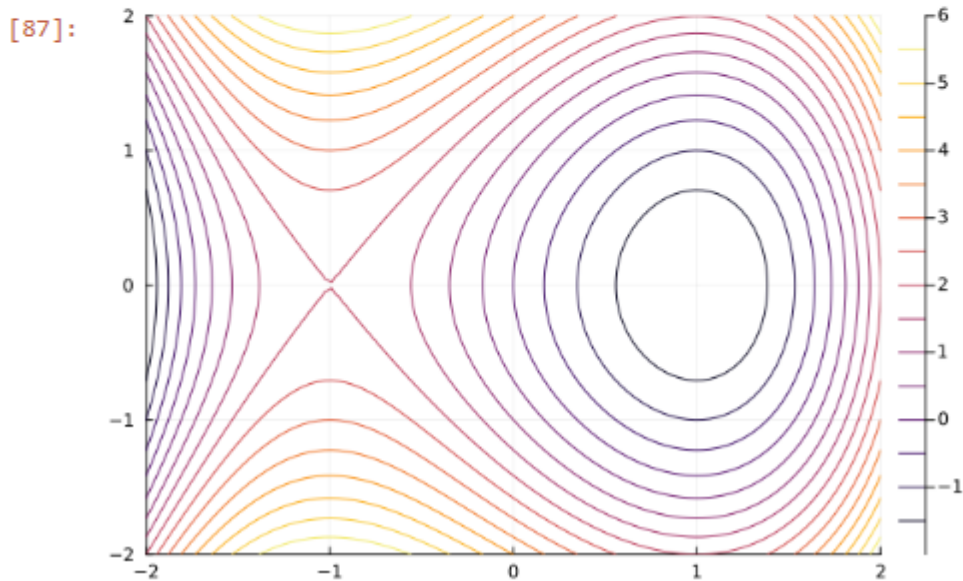


Рис. 24: Линии уровня для функции $h(x, y) = x^3 - 3x + y^2$

Векторное поле можно охарактеризовать векторными линиями. Каждая точка векторной линии является началом вектора поля, который лежит на касательной в данной точке.

Для нахождения векторной линии требуется решить дифференциальное уравнение.

2.11 Анимация

Технически анимированное изображение представляет собой несколько наложенных изображений (или построенных в разных точках графиках) в одном файле.

В Julia рекомендуется использовать gif-анимацию в `pyplot()`.

Строим поверхность (рис. 25):

```
[99]: i = 0  
X = Y = range(-5, stop=5, length=40)  
surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
```

[99]:

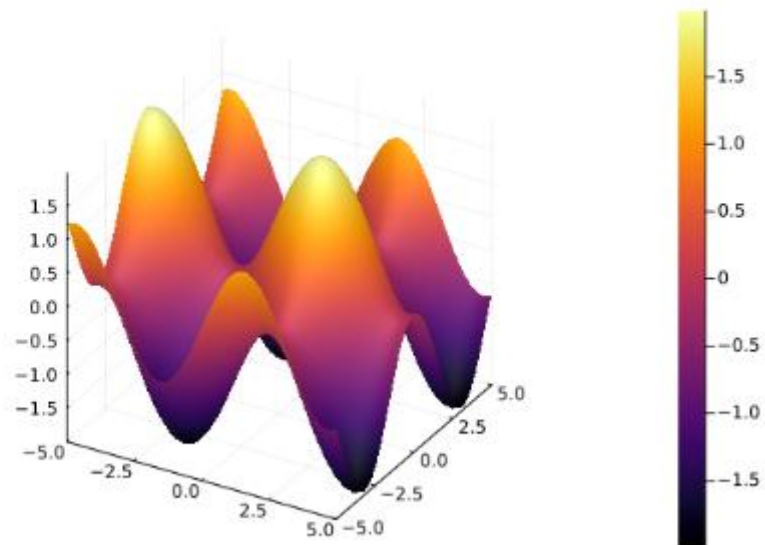


Рис. 25: Статичный график поверхности

Добавляем анимацию (рис. 26):


```
[100]: X = Y = range(-5, stop=5, length=40)
@gif for i in range(0, stop=2pi, length=100)
surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
end
```

[Info: Saved animation to C:\Users\Пользователь\tmp.gif

[100]:

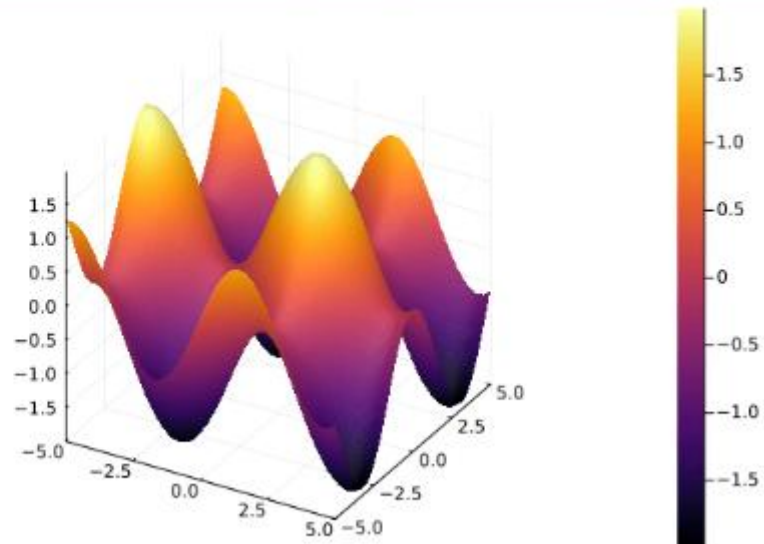


Рис. 26: Анимированный график поверхности

2.12 Гипоциклоида

Гипоциклоида — плоская кривая, образуемая точкой окружности, катящейся по внутренней стороне другой окружности без скольжения.

Построим большую окружность (рис. 27):

```
[104]: # радиус малой окружности:
R = 1
# коэффициент для построения большой окружности:
k = 3
# число отсчётов:
n = 100

# массив значений угла  $\theta$ :
# theta from 0 to 2pi ( + a little extra)
 $\theta$  = collect(0:2*pi/100:2*pi+2*pi/100)
# массивы значений координат:
X = R*k*cos.( $\theta$ )
Y = R*k*sin.( $\theta$ )

# задаём оси координат:
plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1, legend=false, framestyle=:origin)

# большая окружность:
plot!(plt, X,Y, c=:blue, legend=false)
```

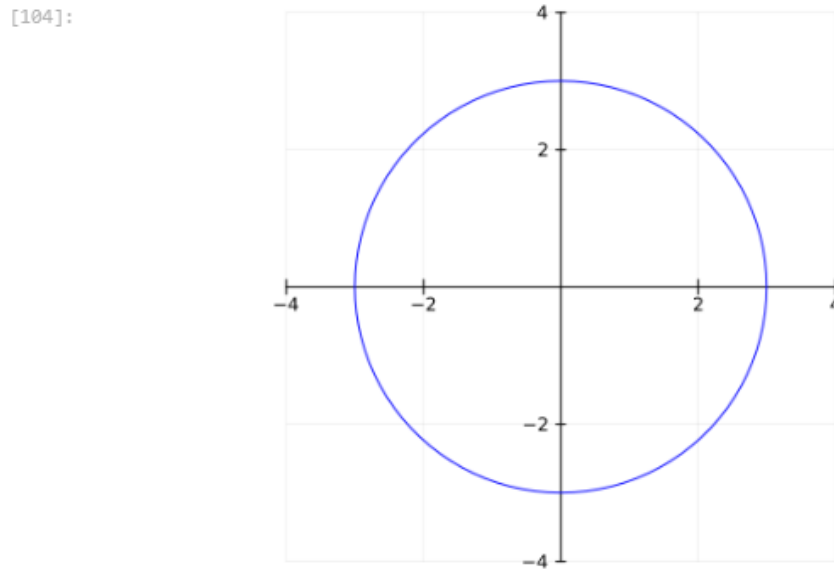


Рис. 27: Большая окружность гипоциклоиды

Для частичного построения гипоциклоиды будем менять параметр t (рис. 28):

```
[106]: i = 50  
t = 0[1:i]  
# гипоциклоида:  
x = R*(k-1)*cos.(t) + R*cos.((k-1)*t)  
y = R*(k-1)*sin.(t) - R*sin.((k-1)*t)  
plot!(x,y, c=:red)
```

[106]:

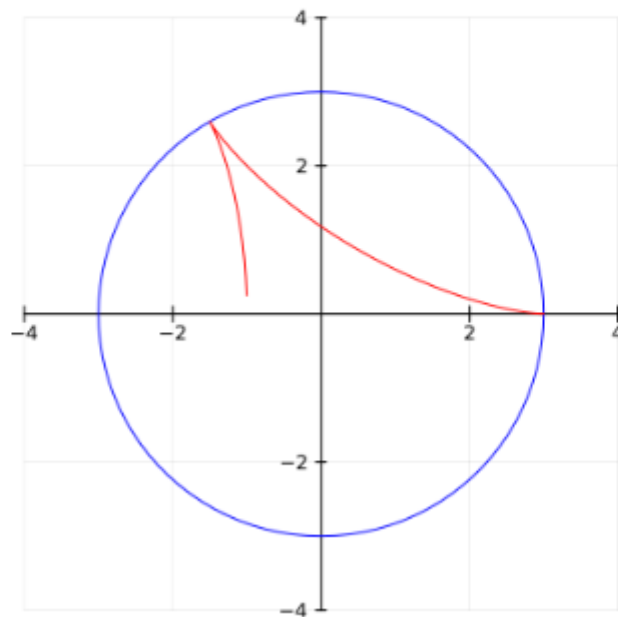


Рис. 28: Половина пути гипоциклоиды

Добавляем малую окружность гипоциклоиды (рис. 29):

```
[107]: # малая окружность:  
xc = R*(k-1)*cos(t[end]) .+ R*cos.(θ)  
yc = R*(k-1)*sin(t[end]) .+ R*sin.(θ)  
plot!(xc,yc,c=:black)
```

[107]:

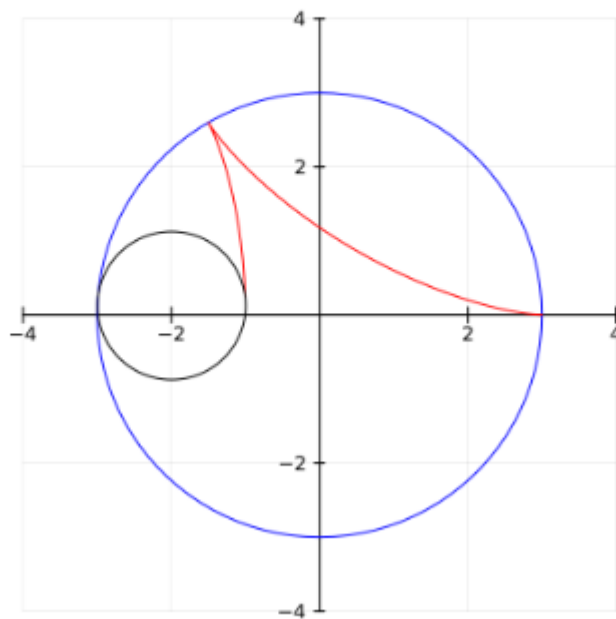


Рис. 29: Малая окружность гипоциклоиды

Добавим радиус для малой окружности (рис. 30):

```
[108]: # радиус малой окружности:
x1 = transpose([R*(k-1)*cos(t[end]) x[end]])
y1 = transpose([R*(k-1)*sin(t[end]) y[end]])
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
```

[108]:

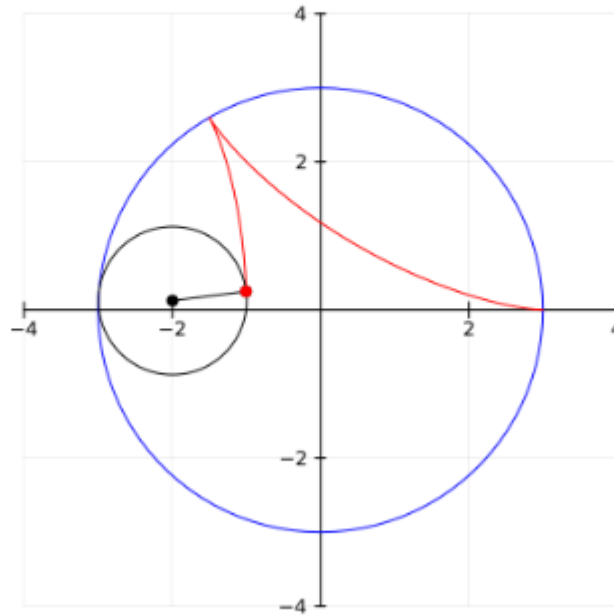


Рис. 30: Малая окружность гипоциклоиды с добавлением радиуса

В конце сделаем анимацию получившегося изображения (рис. 31):

```
[110]: anim = @animate for i in 1:n
# задаём оси координат:
plt=plot(5,xlim=(-4,4),ylim=(-4,4), c=:red, aspect_ratio=1,legend=false, framestyle=:origin)
# большая окружность:
plot!(plt, X,Y, c=:blue, legend=false)
t = 0[1:i]
# гипоциклоида:
x = R*(k-1)*cos.(t) + R*cos.((k-1)*t)
y = R*(k-1)*sin.(t) - R*sin.((k-1)*t)
plot!(x,y, c=:red)
# малая окружность:
xc = R*(k-1)*cos(t[end]) .+ R*cos.(θ)
yc = R*(k-1)*sin(t[end]) .+ R*sin.(θ)
plot!(xc,yc,c=:black)
# радиус малой окружности:
x1 = transpose([R*(k-1)*cos(t[end]) x[end]])
y1 = transpose([R*(k-1)*sin(t[end]) y[end]])
plot!(x1,y1,markershape=:circle,markersize=4,c=:black)
scatter!([x[end]], [y[end]], c=:red, markerstrokecolor=:red)
end

gif(anim,"hypocycloid.gif")
```

[Info: Saved animation to C:\Users\Пользователь\hypocycloid.gif

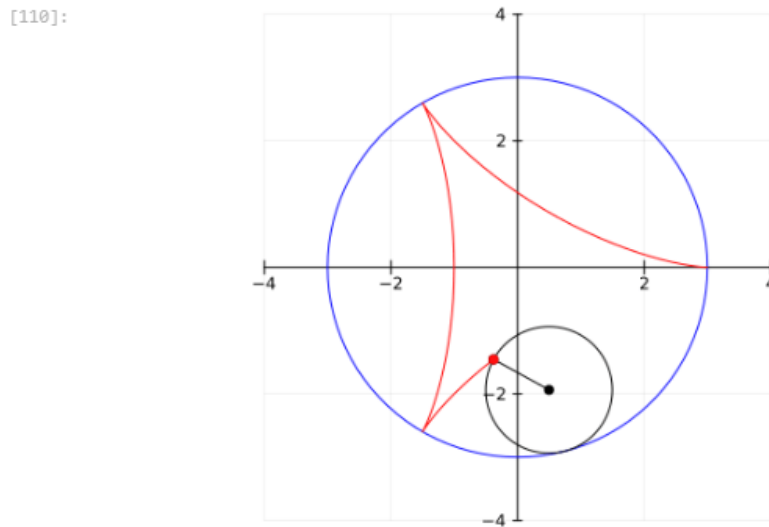


Рис. 31: Малая окружность гипоциклоиды с добавлением радиуса

2.13 Errorbars

В исследованиях часто требуется изобразить графики погрешностей измерения.

Построим график исходных значений (рис. 32):

```
[114]: sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]
```

```
n = 10
```

```
y = [mean(sd*randn(n)) for sd in sds]
```

```
errs = 1.96 * sds / sqrt(n)
```

```
plot(y, ylims = (-1,1))
```

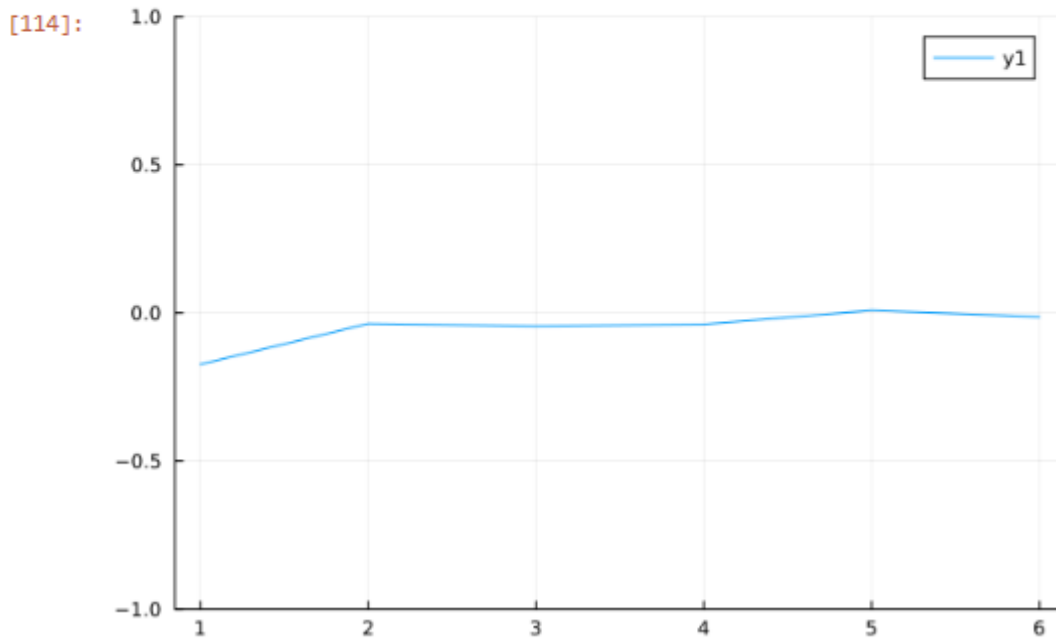


Рис. 32: График исходных значений

Построим график отклонений от исходных значений (рис. 33):

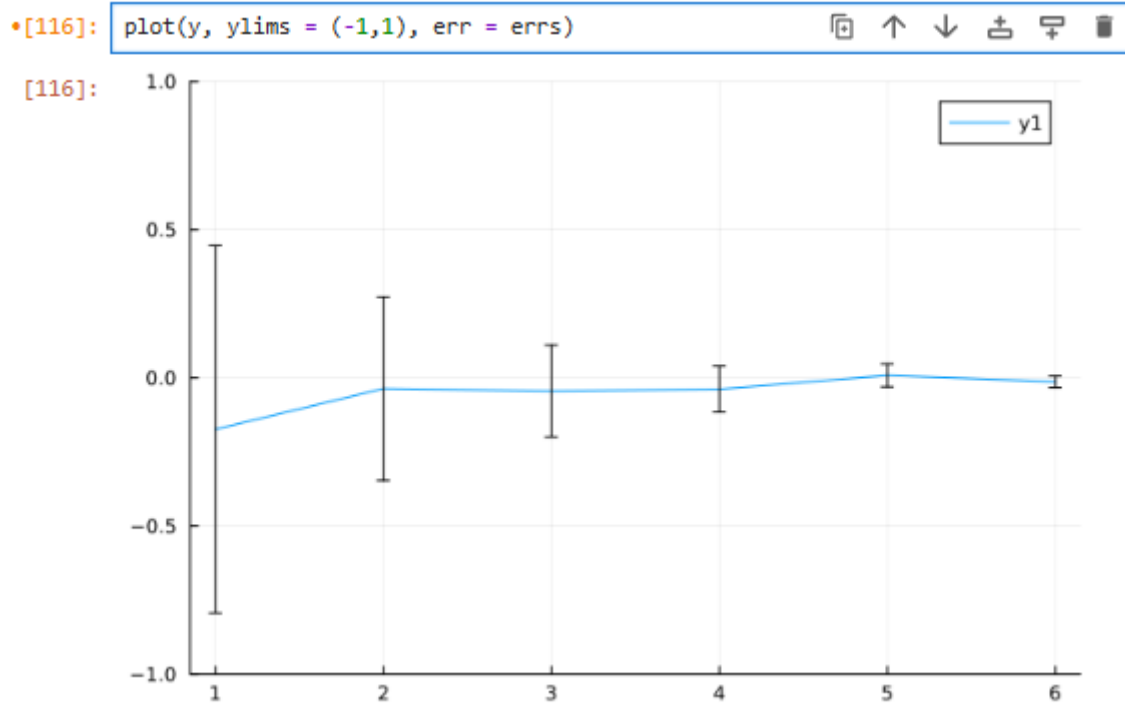


Рис. 33: График исходных значений с отклонениями

Повернём график (рис. 34):

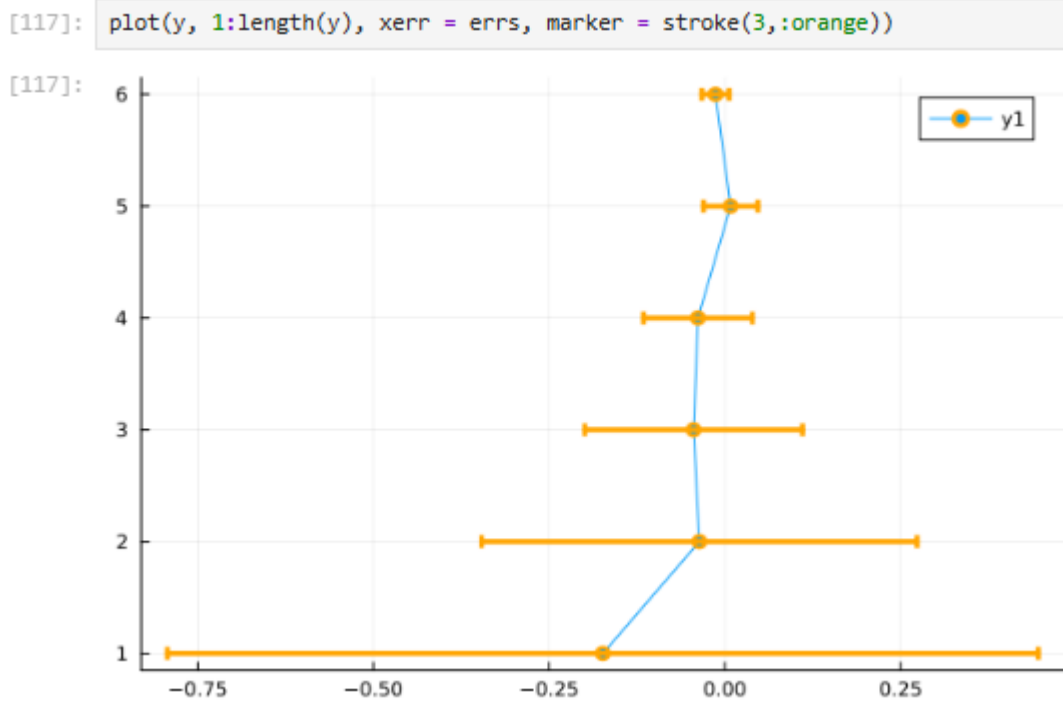


Рис. 34: Поворот графика

Заполним область цветом (рис. 35):


```
[121]: plot(y, ribbon=errs, fill =:cyan)
```

```
[121]:
```

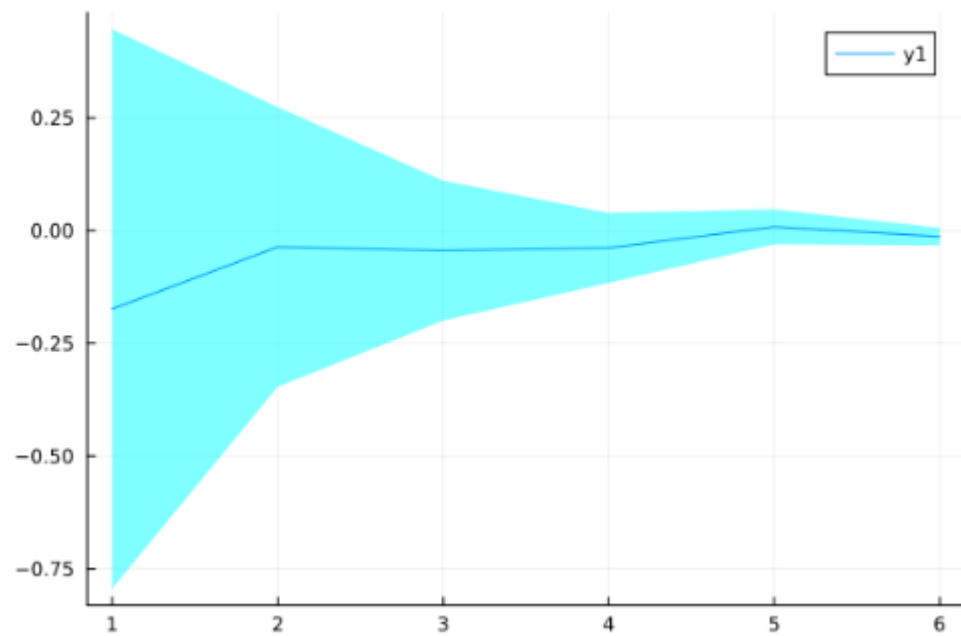
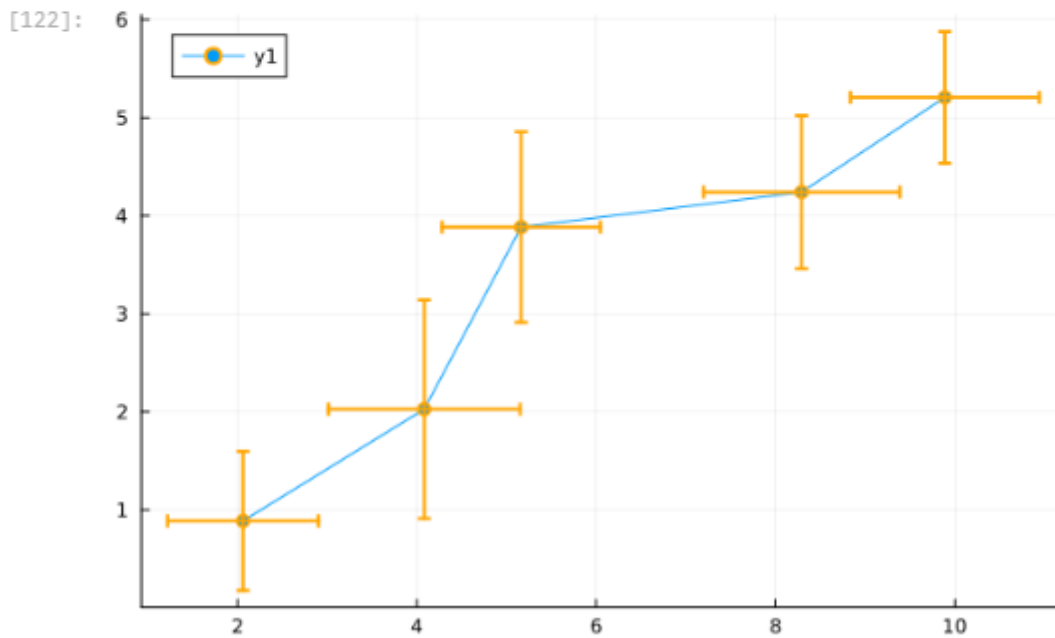


Рис. 35: Заполнение цветом

Можно построить график ошибок по двум осям (рис. 36):

```
[122]: n = 10
x = [(rand()+1) .* randn(n) .+ 2i for i in 1:5]
y = [(rand()+1) .* randn(n) .+ i for i in 1:5]
f(v) = 1.96std(v) / sqrt(n)
xerr = map(f, x)
yerr = map(f, y)
x = map(mean, x)
y = map(mean, y)
plot(x, y,
      xerr = xerr,
      yerr = yerr,
      marker = stroke(2, :orange))
```



ё

Рис. 36: График ошибок по двум осям

Можно построить график асимметричных ошибок по двум осям (рис. 37):

```
[123]: plot(x, y,
           xerr = (0.5xerr, 2xerr),
           yerr = (0.5yerr, 2yerr),
           marker = stroke(2, :orange)
           )
```

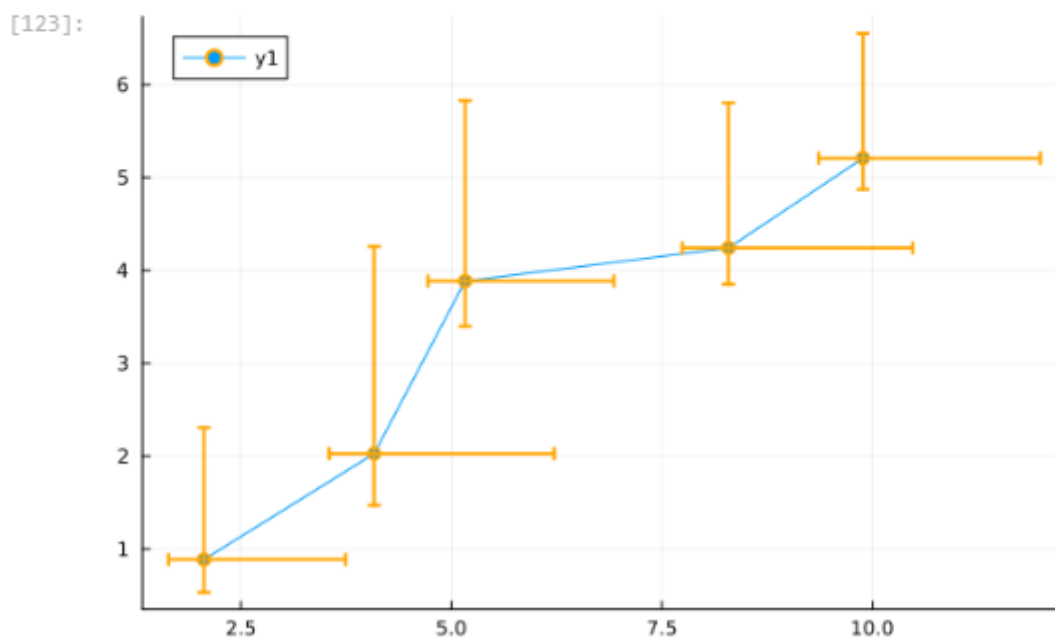


Рис. 37: График асимметричных ошибок по двум осям

2.14 Использование пакета Distributions

Строим гистограмму (рис. 38):

```
[134]: using Plots
      gr()      # включаем GR вместо PyPlot

      using Distributions
      ages = rand(15:55, 1000)

      histogram(ages)
```

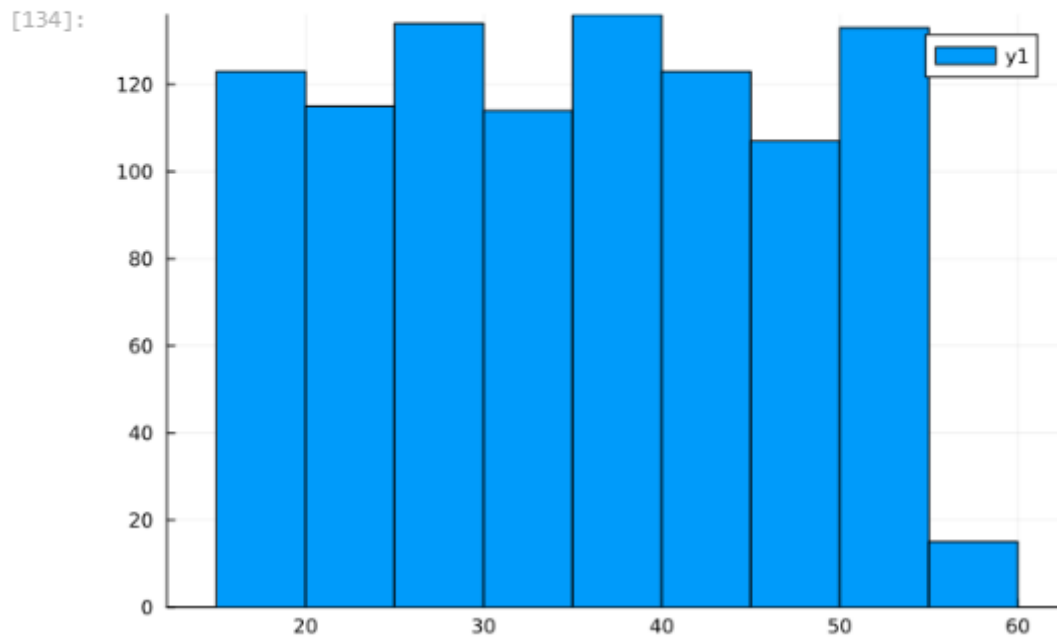


Рис. 38: Гистограмма, построенная по массиву случайных чисел

Задаём нормальное распределение и строим гистограмму (рис. 39):

```
[126]: d=Normal(35.0,10.0)
ages = rand(d,1000)
histogram(
ages,
label="Распределение по возрастам (года)",
xlabel = "Возраст (лет)",
ylabel= "Количество")
```

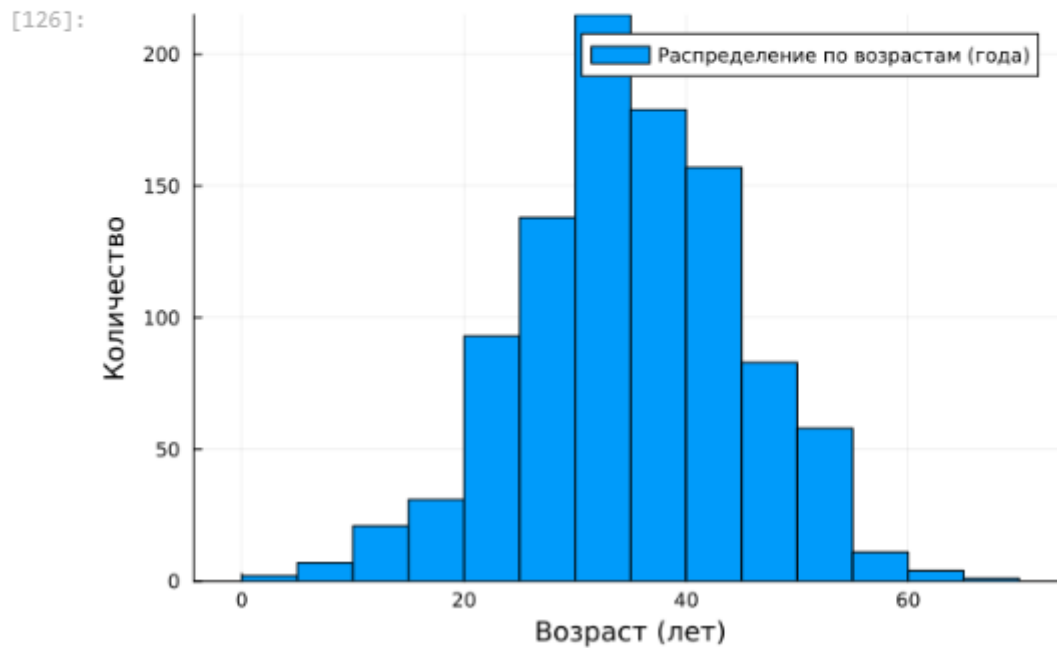


Рис. 39: Гистограмма нормального распределения

Далее применим для построения нескольких гистограмм распределения людей по возрастам на одном графике plotly() (рис. 40):

```
[89]: plotly()
      d1=Normal(10.0,5.0);
      d2=Normal(35.0,10.0);
      d3=Normal(60.0,5.0);
      N=1000;
      ages = (Float64)[];
      ages = append!(ages,rand(d1,Int64(ceil(N/2))));
      ages = append!(ages,rand(d2,N));
      ages = append!(ages,rand(d3,Int64(ceil(N/3))));
      histogram(
        ages,
        bins=50,
        label="Распределение по возрастам (года)",
        xlabel = "Возраст (лет)",
        ylabel= "Количество",
        title = "Распределение по возрастам (года)"
      )
```



```
[144]: using Plots
gr() # Используем GR вместо PyPlot

# построение серии графиков:
x = range(-2, 2, length=10)
y = rand(10, 4)
plot(x, y, layout=(4, 1))
```

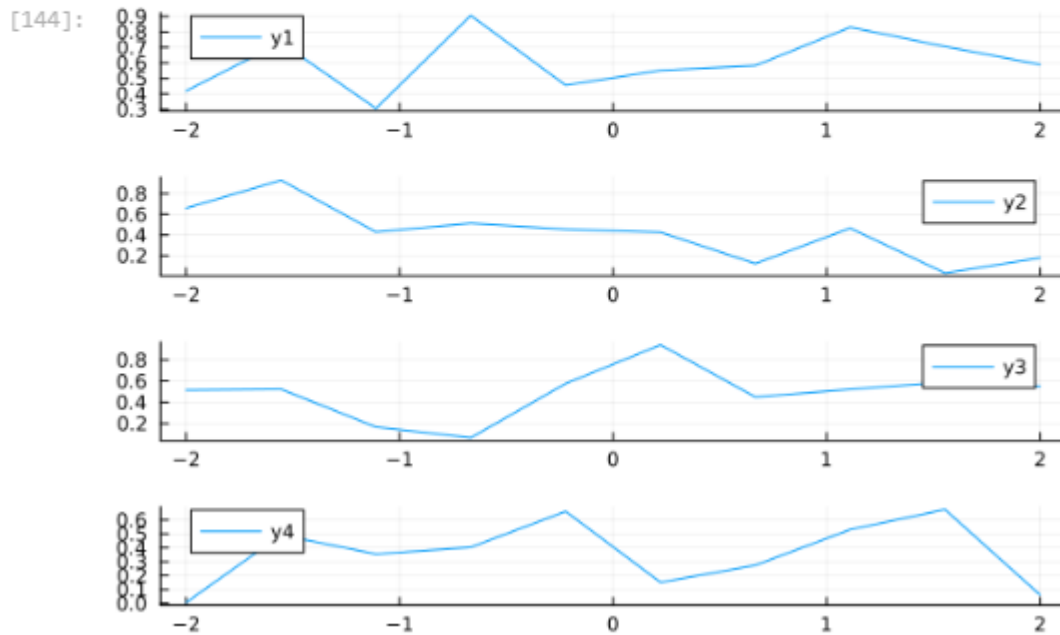


Рис. 40: Гистограмма распределения людей по возрастам

2.15 Подграфики

Определим макет расположения графиков. Команда `layout` принимает кортеж `layout = (N, M)`, который строит сетку графиков $N \times M$. Например, если задать `layout = (4, 1)` на графике четыре серии, то получим четыре ряда графиков (рис. 41):

```
[144]: using Plots
gr() # Используем GR вместо PyPlot

# построение серии графиков:
x = range(-2, 2, length=10)
y = rand(10, 4)
plot(x, y, layout=(4, 1))
```

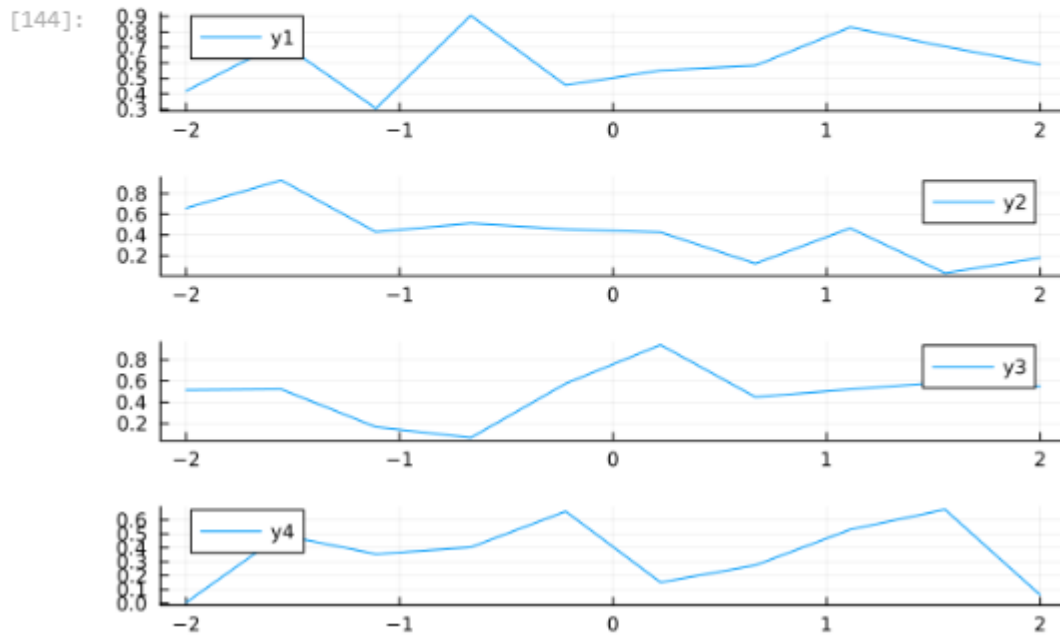


Рис. 41: Серия из 4-х графиков в ряд

Для автоматического вычисления сетки необходимо передать layout целое число (рис. 42):

•[145]: `plot(x,y, [layout=4])`

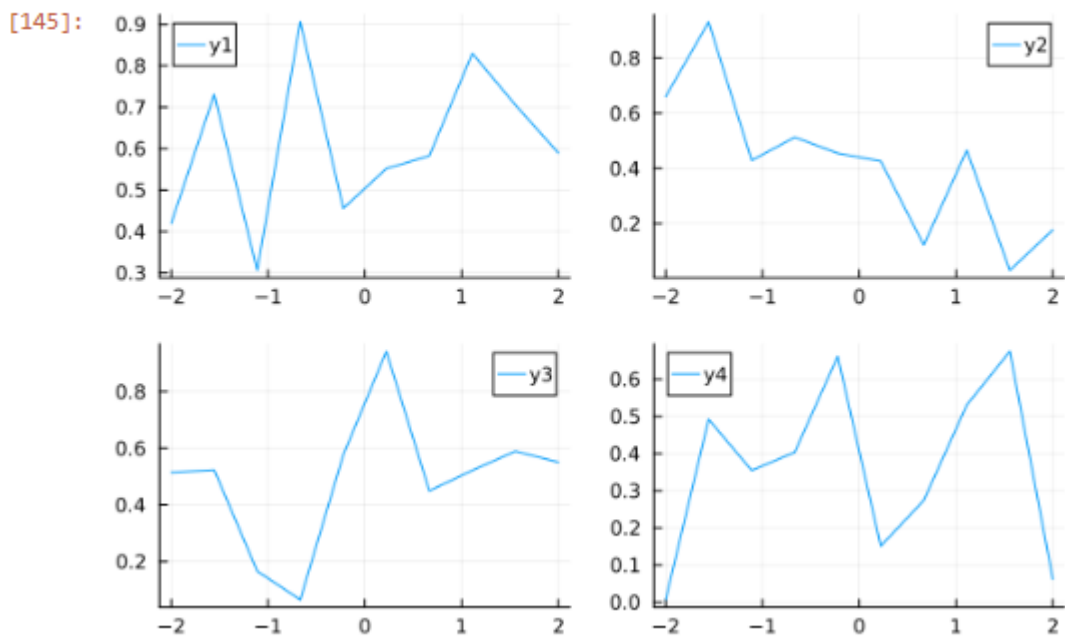


Рис. 42: Серия из 4-х графиков в сетке

Аргумент `heights` принимает в качестве входных данных массив с долями желаемых высот. Если в сумме дроби не составляют 1,0, то некоторые подзаголовки могут отображаться неправильно.

Можно сгенерировать отдельные графики и объединить их в один, например, в сетке 2 × 2 (рис. 43):

```
[147]: # график в виде линий:
p1 = plot(x,y)
# график в виде точек:
p2 = scatter(x,y)
# график в виде линий с оформлением:
p3 = plot(x,y[:,1:2],xlabel="Labelled plot of two columns",lw=2,title="Wide lines")
# 4 гистограммы:
p4 = histogram(x,y)
plot(
p1,p2,p3,p4,
layout=(2,2),
legend=False,
size=(800,600),
background_color = :ivory
)
```

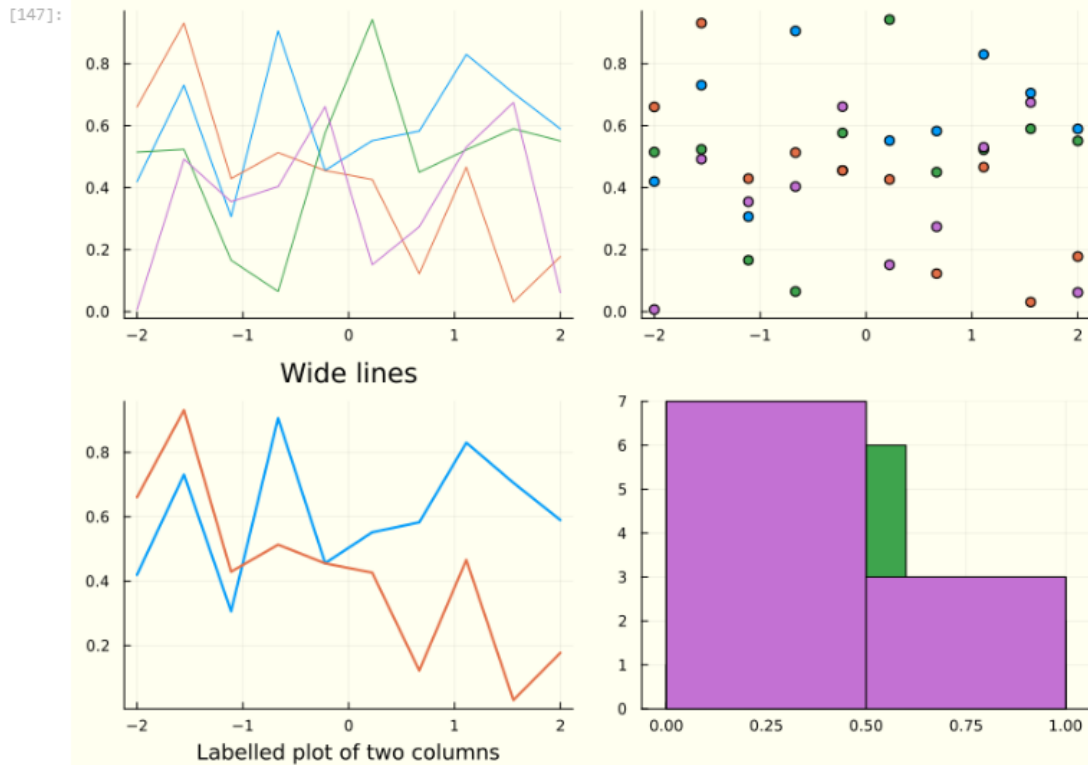


Рис. 43: Объединение нескольких графиков в одной сетке

Обратите внимание, что атрибуты на отдельных графиках применяются к отдельным графикам, в то время как атрибуты в последнем вызове `plot` применяются ко всем графикам.

Разнообразные варианты представления данных (рис. 44):

```
[148]: seriestypes = [:step, :sticks, :bar, :hline, :vline, :path]
titles = ["step" "sticks" "bar" "hline" "vline" "path"]
plot(rand(20,1), st = seriestypes,
layout = (2,3),
ticks=nothing,
legend=false,
title=titles,
m=3
)
```

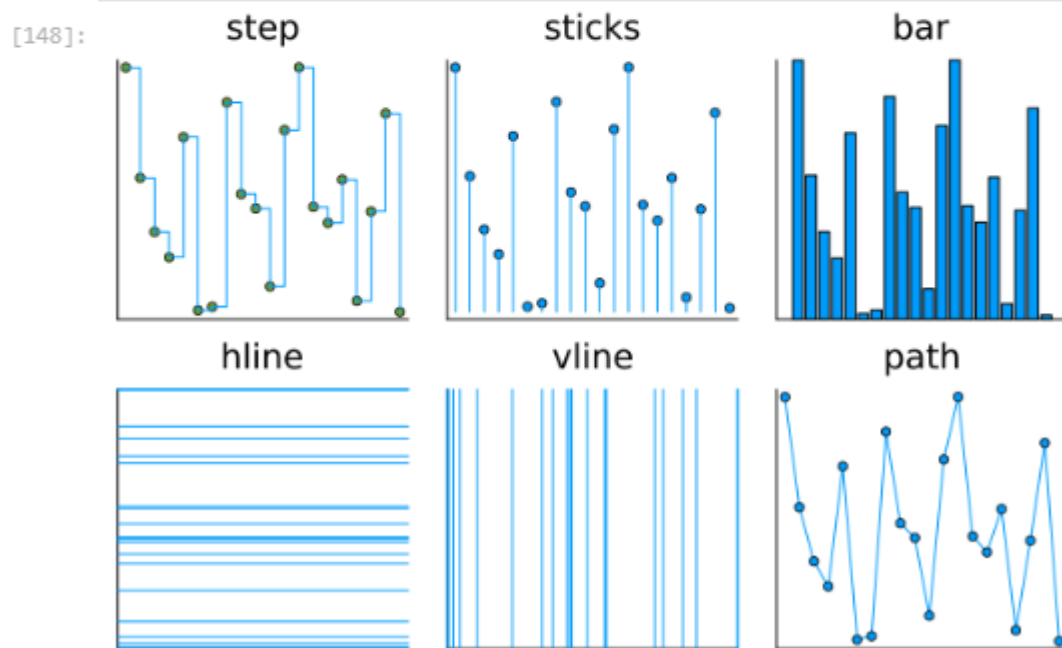


Рис. 44: Разнообразные варианты представления данных

Применение макроса **[layout?]** наиболее простой способ определения сложных макетов. Точные размеры могут быть заданы с помощью фигурных скобок, в противном случае пространство будет поровну разделено между графиками (рис. 45):

```
[149]: l = @layout [ a{0.3w} [grid(3,3)
b{0.2h} ]]
plot(
rand(10,11),
layout = l, legend = false, seriestype = [:bar :scatter :path],
title = ["($i)" for j = 1:1, i=1:11], titleloc = :right, titlefont = font(8)
)
```

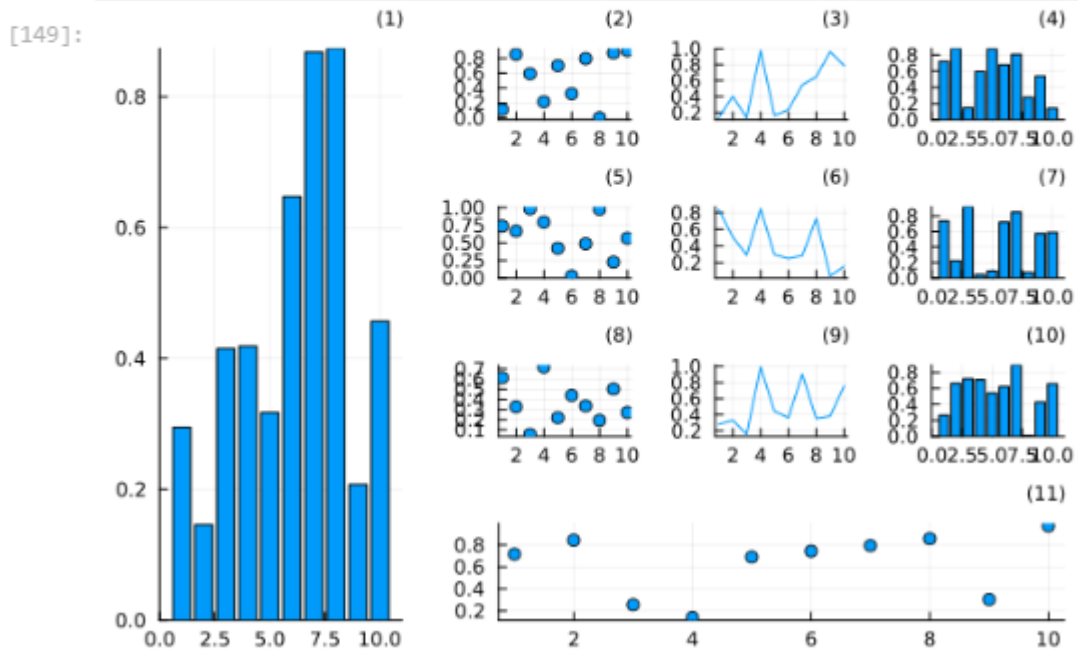


Рис. 45: Демонстрация применения сложного макета для построения графиков

2.16 Самостоятельное выполнение

Выполнение задания №1 (рис. 46):

```
[155]: using Plots
gr()

# Определим диапазон x от 0 до 2π
x = 0:0.01:2π
# Определим функцию y = sin(x)
y = sin.(x)

# Создаём несколько подграфиков для различных типов графиков
plot(layout=(2, 2), size=(800, 600))

# Линейный график
plot!(x, y, seriestype=:line, label="Line", subplot=1, linewidth=2, title="Линейный график")

# Точечный график (используем каждую 10-ю точку для лучшего отображения)
plot!(x[1:10:end], y[1:10:end], seriestype=:scatter, label="Scatter", subplot=2, title="Точечный график")

# Гистограмма
histogram!(y, bins=30, label="Histogram", subplot=3, title="Гистограмма")

# График ступеней
plot!(x, y, seriestype=:step, label="Step", subplot=4, linewidth=2, title="Ступенчатый график")
```

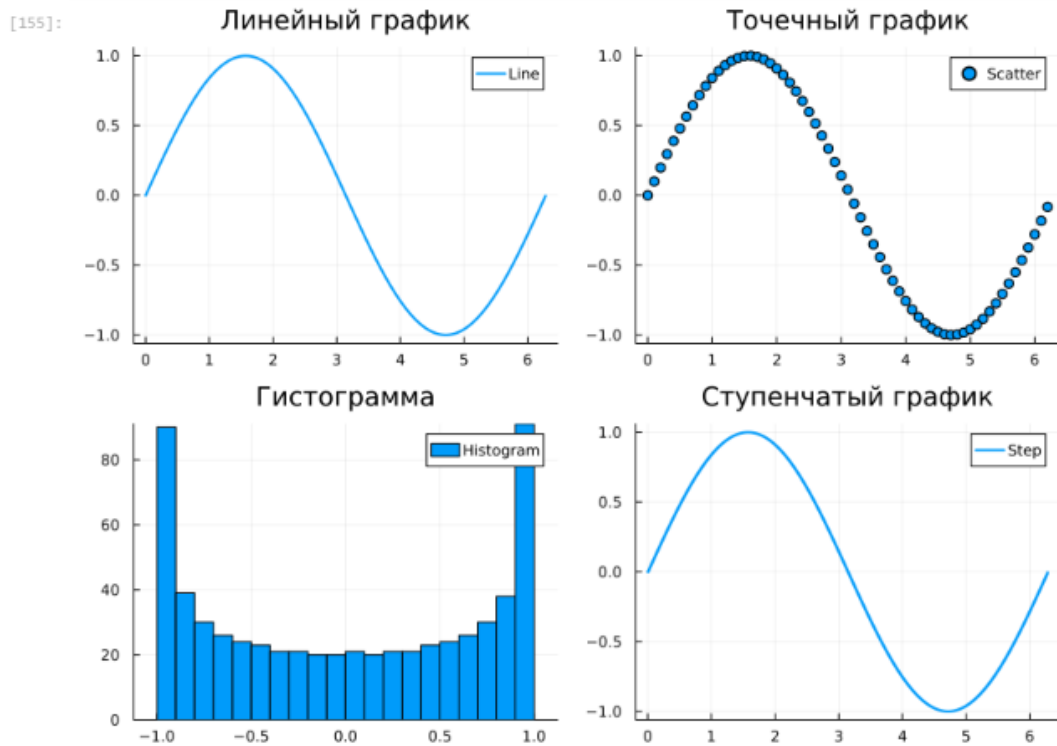


Рис. 46: Решение задания №1

Выполнение задания №2 (рис. 47):

```

]: using Plots
gr()

# Определим диапазон x от 0 до 2π
x = 0:0.01:2π
# Определим функцию y = sin(x)
y = sin.(x)

# Список стилей линий
line_styles = [:solid, :dash, :dot, :dashdot]

# Построим графики с разными стилями линий в одном окне
plt = plot(x, y, linestyle=:solid, label=":solid", xlabel="x", ylabel="y",
           title="Графики функции y = sin(x) с разными стилями линий", linewidth=2)

for ls in line_styles[2:end]
    plot!(plt, x, y, linestyle=ls, label="$ls", linewidth=2)
end

display(plt)

```

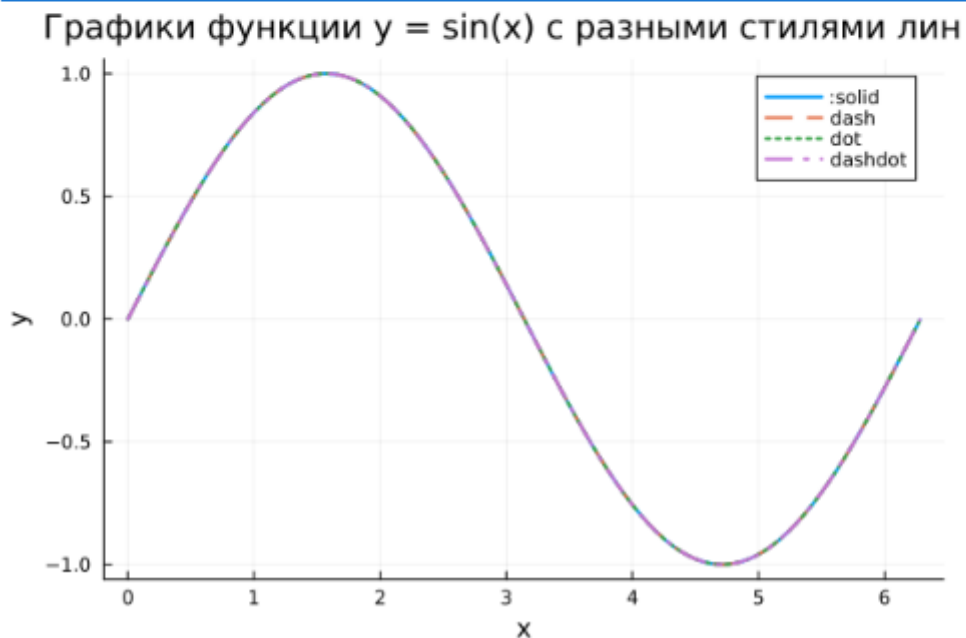


Рис. 47: Решение задания №2

Выполнение задания №3 (рис. 48):

```
[179]: using Plots
gr()

# Определяем функцию y(x)
j(x) = π * x^2 * log(x)

# Определяем диапазон x
x = 0.1:0.01:10

# Построение графика
plot(x, j.(x),
     color = :red,           # Цвет графика
     label = "y(x) = πx²ln(x)", # Легенда (без LaTeX)
     xlabel = "x",           # Подпись оси X
     ylabel = "y(x)",        # Подпись оси Y
     framestyle = :box,      # Стилй рамки
     grid = false,           # Убираем сетку
     xticks = 0:2:10,        # Частота отметок на оси X
     yticks = -50:50:200,    # Частота отметок на оси Y
     linewidth = 2,
     legend = :topleft,
     size = (600, 400))
```

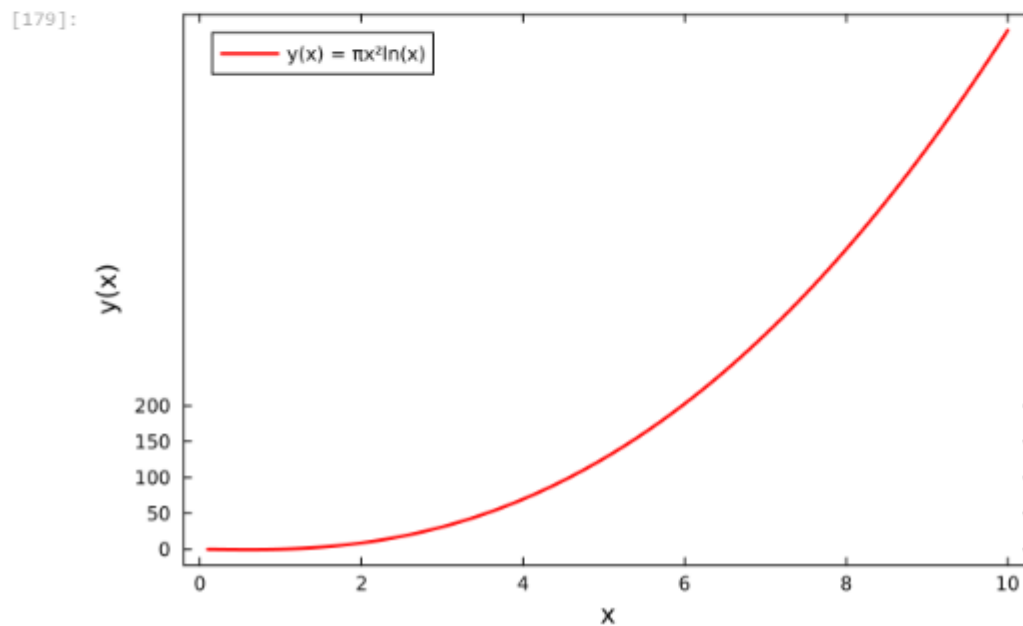


Рис. 48: Решение задания №3

Выполнение задания №4 (рис. 49):

```
[186]: using Plots
gr()

# Задаем вектор x
x = [-2, -1, 0, 1, 2]
# Функция y(x)
J(x) = x.^3 .- 3 .* x
# Вычисляем значения y
y_values = J(x)

# Создаем макет для 4 подграфиков
plt = plot(layout = (2, 2), size = (800, 600))

# График точек
scatter!(plt[1], x, y_values, label = "Точки", title = "Точечный график")

# График линий
plot!(plt[2], x, y_values, label = "Линии", title = "Линейный график")

# График линий и точек
plot!(plt[3], x, y_values, label = "Линии", title = "Линии и точки")
scatter!(plt[3], x, y_values, label = "Точки")

# График кривой (интерполируем для гладкой кривой)
x_smooth = range(-2, 2, length=100)
y_smooth = J(x_smooth)
plot!(plt[4], x_smooth, y_smooth, label = "Кривая", title = "Гладкая кривая")

# Сохраняем график в файл
savefig(plt, "figure_mahorin.png")

[186]: "C:\\Users\\Пользователь\\figure_mahorin.png"
```

Рис. 49: Решение задания №4

Выполнение задания №5 (рис. 50 - рис. 51):


```
[189]: # Задаем вектор x
x = 3:0.1:6
# Определяем функции y1 и y2
y1(x) = pi * x
y2(x) = exp.(x) .* cos.(x)
# 1. Построение графиков на одной оси с легендой и сеткой
plot(x, y1(x), label="y1(x) = x", color=:blue, linewidth=2, grid=true, legend=:topright)
plot!(x, y2(x), label="y2(x) = exp(x) * cos(x)", color=:red, linewidth=2)
# Добавление заголовков
xlabel!("x")
ylabel!("y")
title!("Графики функций y1(x) и y2(x)*")
```

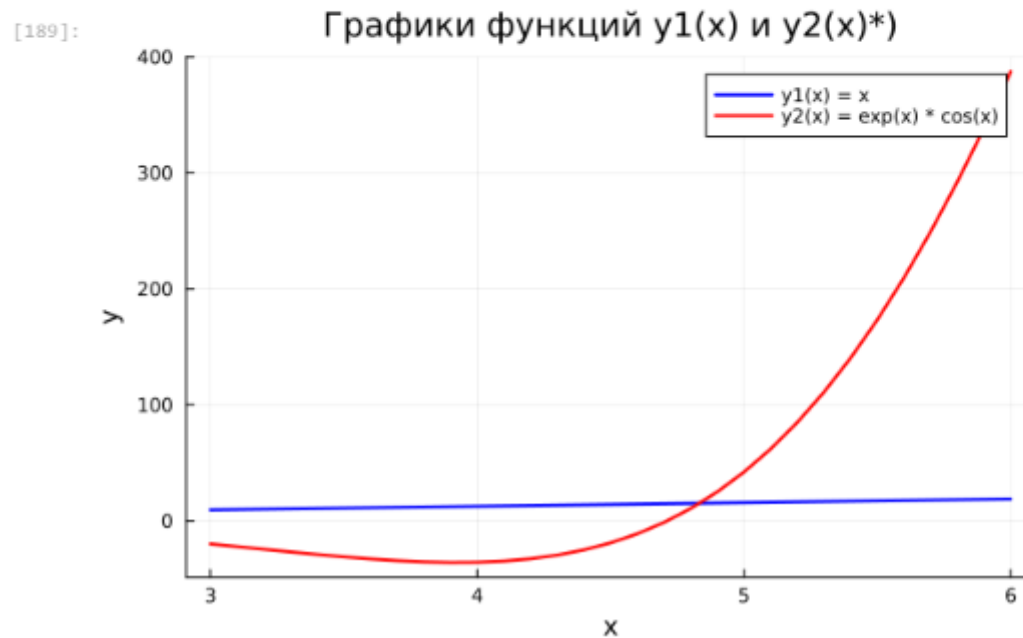


Рис. 50: Решение задания №5

```
[191]: # Построение графиков с двумя осями ординат
p = plot(x, y1(x), label="y1(x) = πx", color=:blue, linewidth=2, grid=true, legend=:topright)
plot!(p, x, y2(x), label="y2(x) = exp(x) * cos (x)", color=:red, linewidth=2)
# Добавляем вторую ось ординат для y2
plot!(p, secondary=true)
# Заголовок и сетка
xlabel!("x")
ylabel!("y1 (primary)", fontsize=10)
ylabel!(p, "y2 (secondary)", fontsize=10)
title!("Графики с двумя осями ординат")
```

[191]:

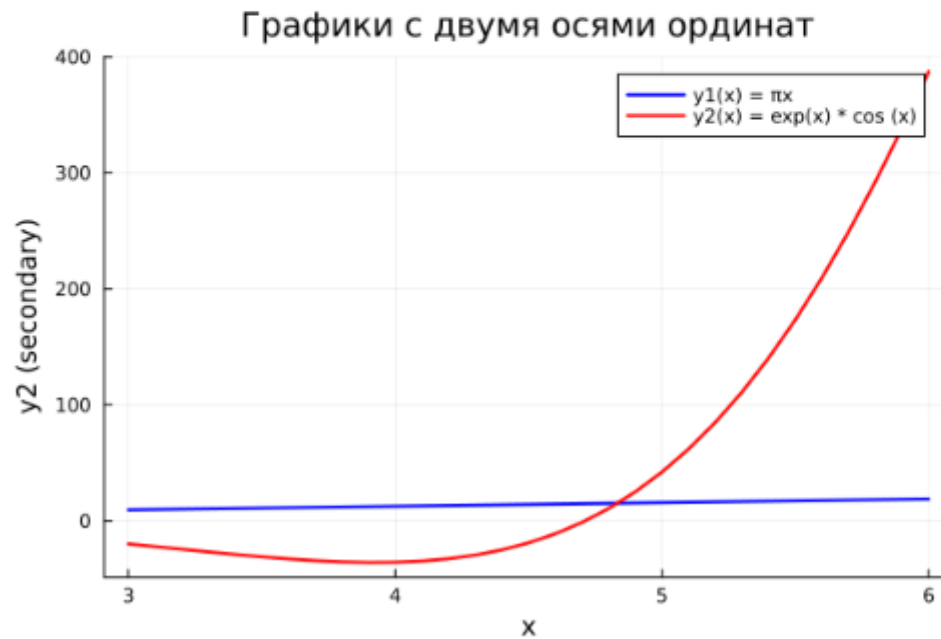


Рис. 51: Решение задания №5

Выполнение задания №6 (рис. 52):

```
[192]: # Шаг 1: Придумываем экспериментальные данные
# Время измерений (например, 10 точек через 1 час)
time = 0:1:9 # Время в часах (от 0 до 9 часов)
# Температурные данные (например, температура варьируется от 20 до 25 градусов)
temperature = 22 .+ 2 .* sin.(time * pi / 5) # Синусоидальное изменение температуры
# Шаг 2: Добавим ошибку измерения (например, стандартное отклонение 0.5 градуса)
# Ошибка измерения - случайные колебания вокруг истинных значений
temperature_error = 0.5 .+ 0.1 .* randn(length(time)) # Ошибка измерений с нормальным распределением
# Шаг 3: Строим график с учетом ошибки измерений
plot(time, temperature, label="Температура", color=:blue, linewidth=2, legend=:topright,
      yerr=temperature_error, marker=:none) # Добавляем подписи осей и заголовок
xlabel!("Время (часы)")
ylabel!("Температура (°C)")
title!("Экспериментальные данные с ошибкой измерения")
```

[192]: Экспериментальные данные с ошибкой измерения

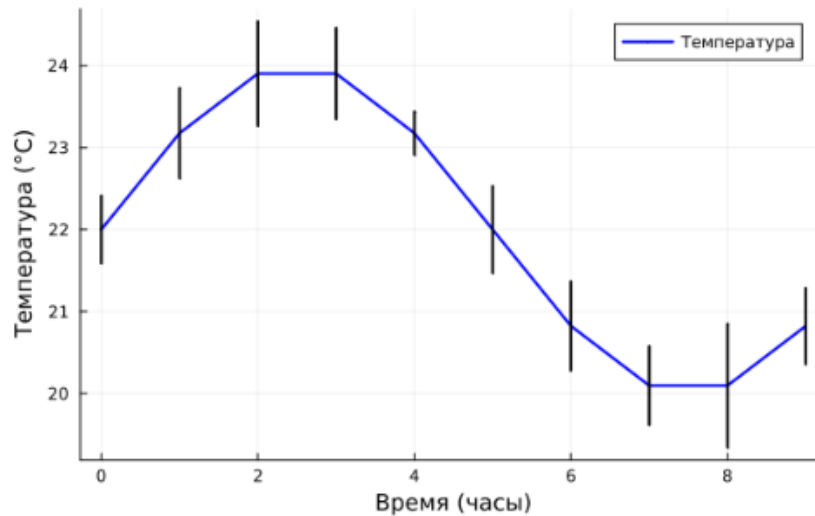


Рис. 52: Решение задания №6

Выполнение задания №7 (рис. 53):

```
[198]: # Генерация случайных данных
x = rand(10) # 10 случайных значений по оси X
y_vals = rand(10) # 10 случайных значений по оси Y
# Построение точечного графика с легендой
scatter(x, y_vals, label="Случайные данные", color=:blue, marker=:o, linewidth=2, legend=:topright)
# Добавление подписей осей и заголовка
xlabel!("X" )
ylabel!("y")
title!("Точечный график случайных данных")
```

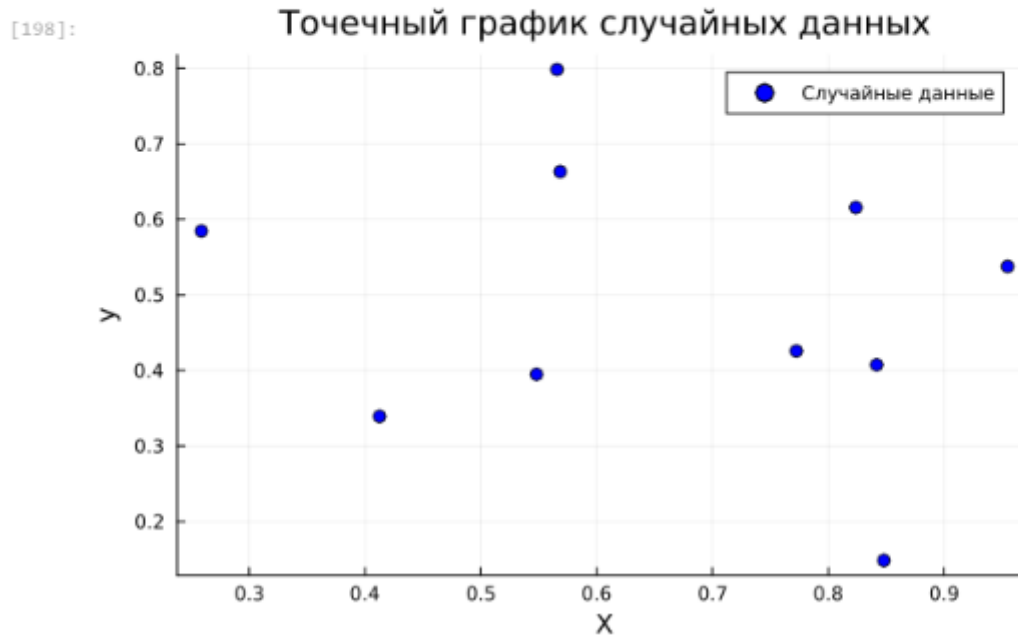


Рис. 53: Решение задания №7

Выполнение задания №8 (рис. 54):

```
[202]: # Генерация случайных данных для 3D графика
x_vals = rand(10) # 10 случайных значений по оси X
y_vals = rand(10) # 10 случайных значений по оси Y
z_vals = rand(10) # 10 случайных значений по оси Z
# Построение 3-мерного точечного графика
scatter3d(x_vals, y_vals, z_vals, label="Случайные данные", color=:blue, marker=:o, linewidth=2)
# Добавление подписей осей и заголовка
xlabel!("Ось X")
ylabel!("Ось Y")
zlabel!("Ось Z")
title!("3-мерный точечный график случайных данных")
```

[202]: 3-мерный точечный график случайных данных

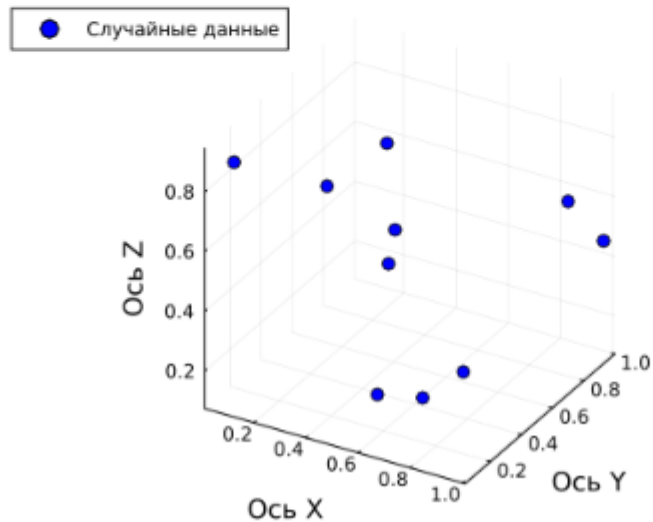


Рис. 54: Решение задания №8

Выполнение задания №9 (рис. 55):

```
[205]: # Подготовим данные для анимации
x = 0:0.1:10 # Диапазон значений аргумента (от 0 до 10)
y_vals = sin.(x) # Значения синусоиды (переименовали y в y_vals)

# Создаем анимацию
anim = @animate for i in 1:length(x)
    plot(x[1:i], y_vals[1:i], label="Синусоиды", color=:blue, linewidth=2)
    xlabel!("x")
    ylabel!("sin(x)")
    title!("Построение синусоиды")
end

# Сохранение анимации в файл
gif(anim, "sin_wave_animation.gif", fps=10)
```

[Info: Saved animation to C:\Users\Пользователь\sin_wave_animation.gif

[205]:

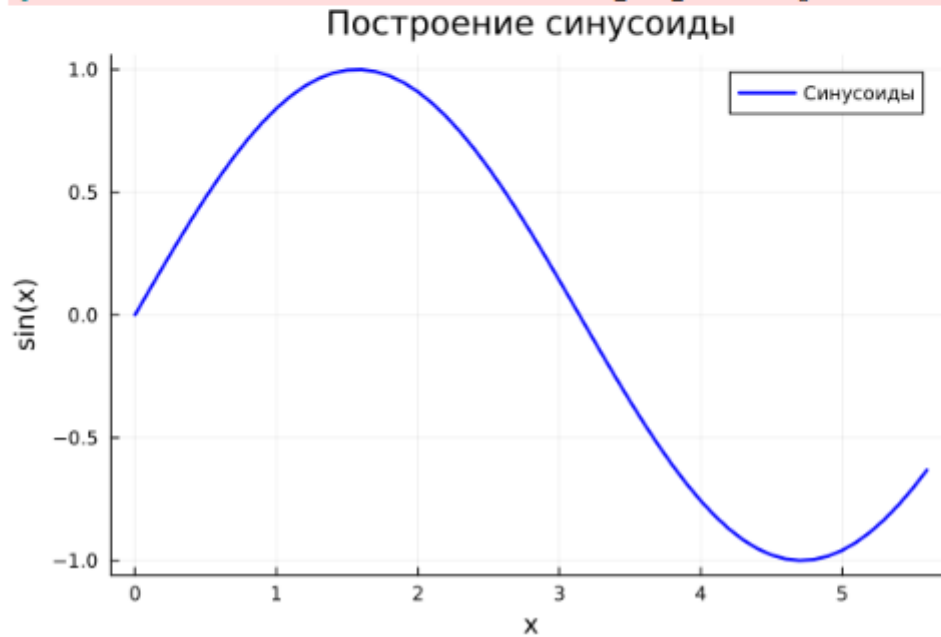


Рис. 55: Решение задания №9

Выполнение задания №10 (рис. 56):

```
[212]: using Plots
gr()

# функция для вычисления координат гипоциклоиды
function hypocycloid(R, r, theta)
    X = (R - r) * cos.(theta) + r * cos.(((R - r) / r) * theta)
    Y = (R - r) * sin.(theta) - r * sin.(((R - r) / r) * theta)
    return X, Y
end

# функция для создания анимации гипоциклоиды для целого значения k
function create_hypocycloid_animation(k)
    R = 10 # Радиус большой окружности
    r = R / k # Радиус маленькой окружности
    theta = 0:0.05:2π # Угол от 0 до 2π, шаг 0.05
    x, y = hypocycloid(R, r, theta)

    anim = @animate for i in 1:length(theta) # итерация по всем углам
        plot(x[1:i], y[1:i],
            label="Гипоциклоида (k=$k)",
            color=:blue,
            linewidth=2,
            xlabel="x",
            ylabel="y",
            title="Анимация гипоциклоиды для k=$k",
            legend=:topright,
            xlims=(-R, R),
            ylims=(-R, R),
            aspect_ratio=:equal)
    end
    # Сохранение анимации в файл
    gif(anim, "hypocycloid_k_$k.gif", fps=10)
end

# Создание анимации для целого значения k = 2
create_hypocycloid_animation(2)
```



Рис. 56: Решение задания №10

Выполнение задания №11 (рис. 57):

```
[220]: using Plots
gr()

# функция для вычисления координат эпициклоиды
function epicycloid(R, r, theta)
    x = (R + r) * cos.(theta) - r * cos.(((R + r) / r) * theta)
    y = (R + r) * sin.(theta) - r * sin.(((R + r) / r) * theta)
    return x, y
end

# функция для создания анимации эпициклоиды для целого значения k
function create_epicycloid_animation(k)
    R = 10 # Радиус большой окружности
    r = R / k # Радиус маленькой окружности
    theta = 0:0.05:2π # Угол от 0 до 2π, шаг 0.05
    x, y = epicycloid(R, r, theta)

    anim = @animate for i in 1:length(theta) # итерация по всем углам
        plot(x[1:i], y[1:i],
            label="Эпициклоида (k=$k)",
            color=:blue,
            linewidth=2,
            xlabel="x",
            ylabel="y",
            title="Анимация эпициклоиды для k=$k",
            legend=:topright,
            xlims=(-(R+2r), R+2r),
            ylims=(-(R+2r), R+2r),
            aspect_ratio=:equal)
    end
    # Сохранение анимации в файл
    gif(anim, "epicycloid_k_$k.gif", fps=10)
end

# Создание анимации для целого значения k = 2
create_epicycloid_animation(2)
```

[Info: Saved animation to C:\Users\Пользователь\epicycloid_k_2.gif

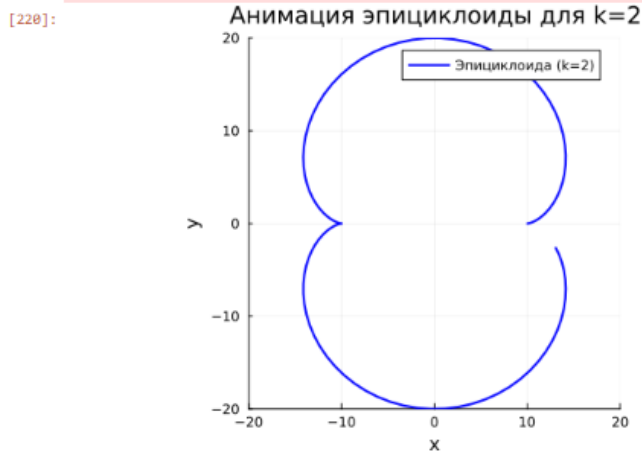


Рис. 57: Решение задания №11

3 Вывод

В ходе выполнения лабораторной работы был освоен синтаксис языка Julia для построения графиков.

4 Список литературы. Библиография

[1] Julia Documentation: <https://docs.julialang.org/en/v1/>